MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

School of Computer Engineering

Manipal – 576 104

Operating Systems Lab [CSS-2211]

Fourth Semester

B. Tech. (CSE Stream)

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

SCHOOL OF COMPUTER ENGINEERING

CERTIFICATE

This is to certify that Ms. Hridini Pandey
Reg. No.: 240911482
Section: ITB
Roll No.: 39
has satisfactorily completed the lab exercises prescribed for OPERATING SYSTEMS LAB [CSS 2211] of Second Year B.Tech. Degree at MIT, Manipal, in the academic year 2025-2026.

Date: …….....
Signature
Faculty in Charge

INDEX

LAB NO: 1

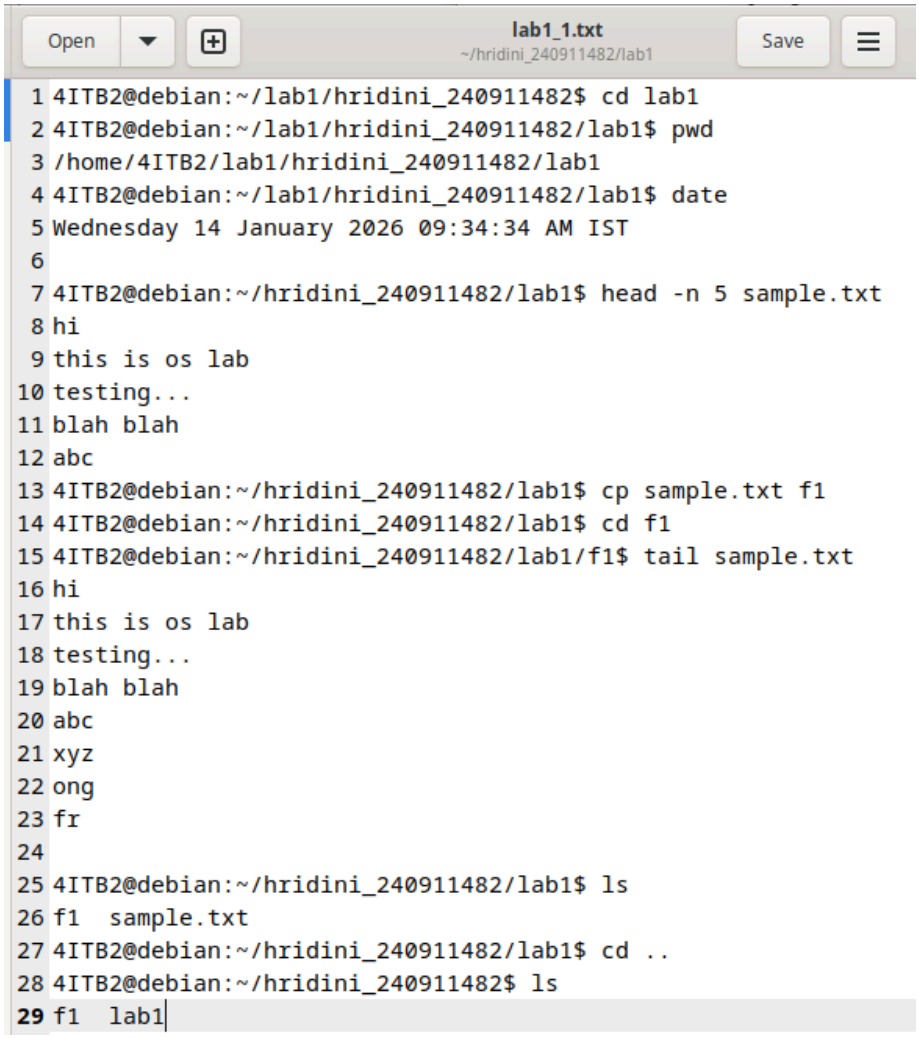
UNIX SHELL COMMANDS

Lab Exercises:

1. Create a folder with your <name >_registration number in the /home folder. Use this
folder to save rest of lab exercises. For each lab create a separate folder ex: Lab1, Lab2…
Lab11.

```
4ITB2@debian:~$ cd hridini_240911482
4ITB2@debian:~/hridini_240911482$ mkdir lab1
4ITB2@debian:~/hridini_240911482$ cd lab1
4ITB2@debian:~/hridini_240911482/lab1$ cat sample.txt
cat: sample.txt: No such file or directory
4ITB2@debian:~/hridini_240911482/lab1$ touch sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ cat sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ head sample.txt
hi
this is os lab
testing...
blah blah
abc
xyz
ong
fr
4ITB2@debian:~/hridini_240911482/lab1$ mkdir f1
4ITB2@debian:~/hridini_240911482/lab1$ cp sample.txt/f1
cp: missing destination file operand after 'sample.txt/f1'
Try 'cp --help' for more information.
4ITB2@debian:~/hridini_240911482/lab1$ cd..
bash: cd..: command not found
4ITB2@debian:~/hridini 240911482/lab1$ mkdir f1
```

2. Execute and write output of any 5 important commands explained so far in this manual. Save the output in Lab1_1.txt . Commands and the output I copied from the command prompt.



3. Explore the following commands along with their various options. (Some of the options are specified in the bracket)

a. cat (variation used to create a new file and append to existing file)

b. head and tail (-n, -c)

c. cp (-n, -i, -f)

d. mv (-f, -i) [try (i) mv dir1 dir2 (ii) mv file1 file2 file3 ... directory]

e. rm (-r, -i, -f)

f. rmdir (-r, -f)

g. find (-name, -type)

Save the output in Lab1_2.txt, Commands and the output I copied from the command prompt.

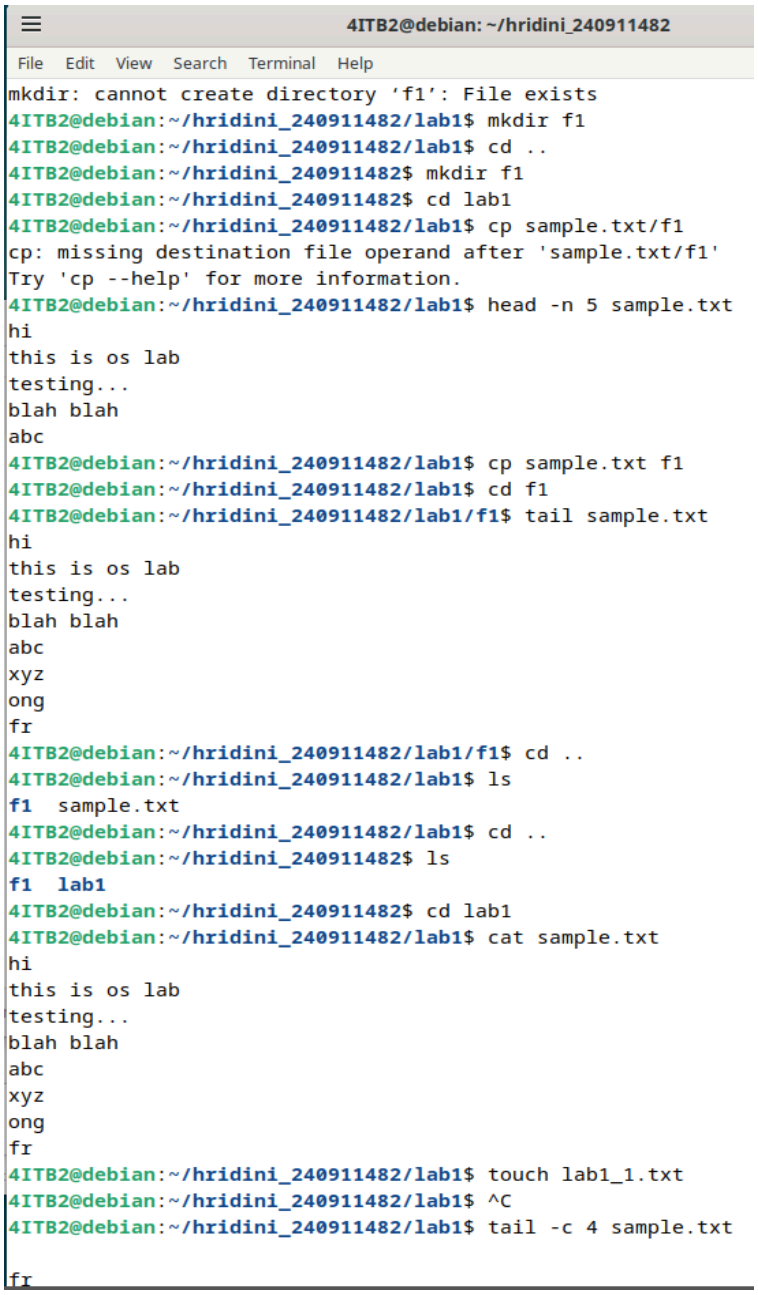
3. List all the file names satisfying following criteria

a. has the extension .txt.

b. containing atleast one digit.

c. having minimum length of 4.

d. does not contain any of the vowels as the start letter.



```
4ITB2@debian: ~/hridini_240911482
File  Edit  View  Search  Terminal  Help
4ITB2@debian:~/hridini_240911482/lab1$ tail -n 5 sample.txt
blah blah
abc
xyz
ong
fr
4ITB2@debian:~/hridini_240911482/lab1$ tail sample.txt
hi
this is os lab
testing...
blah blah
abc
xyz
ong
fr
4ITB2@debian:~/hridini_240911482/lab1$ tail -i 5 sample.txt
tail: invalid option -- 'i'
Try 'tail --help' for more information.
4ITB2@debian:~/hridini_240911482/lab1$ find sample.txt
sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ cd f1
4ITB2@debian:~/hridini_240911482/lab1/f1$ touch ex.txt
4ITB2@debian:~/hridini_240911482/lab1/f1$ cd ..
4ITB2@debian:~/hridini_240911482/lab1$ find ex.txt
find: ‘ex.txt’: No such file or directory
4ITB2@debian:~/hridini_240911482/lab1$ find .-name*.ex
find: ‘.-name*.ex’: No such file or directory
4ITB2@debian:~/hridini_240911482/lab1$ find . -name "ex.txt"
./f1/ex.txt
4ITB2@debian:~/hridini_240911482/lab1$ cd -i sample.txt s2.txt
bash: cd: -i: invalid option
cd: usage: cd [-L|[-P [-e]] [-@]] [dir]
4ITB2@debian:~/hridini_240911482/lab1$ cd -i sample.txt s2.txt
bash: cd: -i: invalid option
cd: usage: cd [-L|[-P [-e]] [-@]] [dir]
4ITB2@debian:~/hridini_240911482/lab1$ cp -i sample.txt s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ cp -i sample.txt s2.txt
cp: overwrite 's2.txt'? n
4ITB2@debian:~/hridini_240911482/lab1$ cp -n sample.txt s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ cp -n sample.txt s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ cp -n lab1_1.txt s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ cp -f lab1_1.txt s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ mv sample.txt f1
4ITB2@debian:~/hridini_240911482/lab1$ cd f1
4ITB2@debian:~/hridini_240911482/lab1/f1$ ls
ex.txt  sample.txt
```


4ITB2@debian: ~/hridini_240911482
File Edit View Search Terminal Help
4ITB2@debian:~/hridini_240911482/lab1$ cd f1
4ITB2@debian:~/hridini_240911482/lab1/f1$ ls
ex.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1/f1$ cd ..
4ITB2@debian:~/hridini_240911482/lab1$ ls
f1 lab1_1.txt s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ rm -i s2.txt
rm: remove regular file 's2.txt'? n
4ITB2@debian:~/hridini_240911482/lab1$ cd f1
4ITB2@debian:~/hridini_240911482/lab1/f1$ mv sample.txt ..
4ITB2@debian:~/hridini_240911482/lab1/f1$ cd ..
4ITB2@debian:~/hridini_240911482/lab1$ ls
f1 lab1_1.txt s2.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ rm -r f1
4ITB2@debian:~/hridini_240911482/lab1$ ls
lab1_1.txt s2.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ ls *.txt
lab1_1.txt s2.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ touch this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls *[0-9]*
lab1_1.txt s2.txt this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls ????*
lab1_1.txt s2.txt sample.txt this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls ???????*
lab1_1.txt sample.txt this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls [!aeiouAEIOU]*
lab1_1.txt s2.txt sample.txt this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls !*.txt
ls [!aeiouAEIOU]*.txt
lab1_1.txt s2.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ ls [!aeiouAEIOU]*.txt
lab1_1.txt s2.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ touch abc.txt
4ITB2@debian:~/hridini_240911482/lab1$ ls
abc.txt lab1_1.txt s2.txt sample.txt this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls [!aeiouAEIOU]*.txt
lab1_1.txt s2.txt sample.txt
4ITB2@debian:~/hridini_240911482/lab1$ ls !(*.txt)
this123.c
4ITB2@debian:~/hridini_240911482/lab1$ ls ??????
s2.txt
4ITB2@debian:~/hridini_240911482/lab1$ cd ..
4ITB2@debian:~/hridini_240911482$ cd lab1
4ITB2@debian:~/hridini_240911482/lab1$ touch lab1.txt
4ITB2@debian:~/hridini_240911482/lab1$ cd ..
4ITB2@debian:~/hridini_240911482$

LAB NO: 2

ADVANCED UNIX SHELL COMMANDS

Lab Exercises

1. Execute all the commands explained in this section and write the output.

```
4ITB2@debian:~/hridini_240911482/lab2$ touch fruitlist.txt
4ITB2@debian:~/hridini_240911482/lab2$ grep apple fruitlist.txt
apple
pineapple
fruit-apple
4ITB2@debian:~/hridini_240911482/lab2$ wc fruitlist.txt
11 10 73 fruitlist.txt
4ITB2@debian:~/hridini_240911482/lab2$ sort fruitlist.txt

apple
banana
fruit-apple
grape
mango
orange
peach
pear
pineapple
tomato
```

```
4ITB2@debian:~/hridini_240911482/lab2$ grep -v apple fruitlist.txt
grape
orange
banana
pear
peach
mango
tomato

4ITB2@debian:~/hridini_240911482/lab2$ grep -x apple fruitslist.txt
grep: fruitslist.txt: No such file or directory
4ITB2@debian:~/hridini_240911482/lab2$ grep -x apple fruitlist.txt
apple
4ITB2@debian:~/hridini_240911482/lab2$ grep ^p fruitlist.txt
pineapple
pear
peach
```

```
4ITB2@debian:~/hridini_240911482/lab2$ sort -r fruitlist.txt
tomato
pineapple
pear
peach
orange
mango
grape
fruit-apple
banana
apple

4ITB2@debian:~/hridini_240911482/lab2$ cut -c1-3 fruitlist.txt
app
gra
ora
pin
fru
ban
pea
pea
man
tom
```

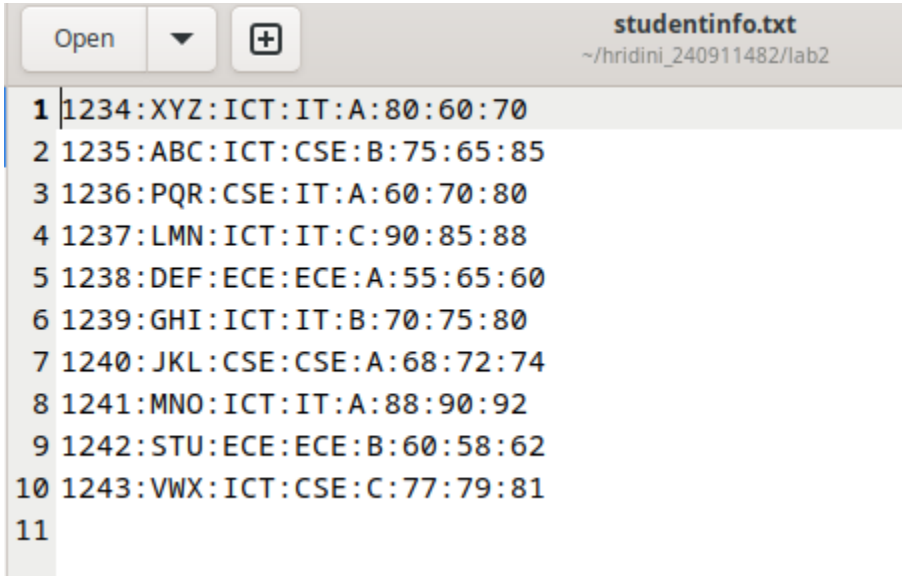
```
4ITB2@debian:~/hridini_240911482/lab2$ ls -l | sed -n '/^d/p'
4ITB2@debian:~/hridini_240911482/lab2$ ps
    PID TTY          TIME CMD
   7846 pts/0    00:00:00 bash
 408038 pts/0    00:00:00 ps
4ITB2@debian:~/hridini_240911482/lab2$ tr 'a-z' 'A-Z' < fruitlist.txt
ps
APPLE
GRAPE
ORANGE
PINEAPPLE
FRUIT-APPLE
BANANA
PEAR
PEACH
MANGO
TOMATO
```

```
TOMATO

    PID TTY          TIME CMD
   7846 pts/0    00:00:00 bash
 408040 pts/0    00:00:00 ps
4ITB2@debian:~/hridini_240911482/lab2$ echo "2^5" | bc
32
4ITB2@debian:~/hridini_240911482/lab2$ sort fruitlist.txt

apple
banana
fruit-apple
grape
mango
orange
peach
pear
pineapple
tomato
```

2. Write grep commands to do the following activities:
• To select the lines from a file that have exactly two characters.
• To select the lines from a file that start with the upper case letter.
• To select the lines from a file that end with a period.
• To select the lines in a file that has one or more blank spaces.
• To select the lines in a file and direct them to another file which has digits as
one of the characters in that line.

```
4ITB2@debian:~/hridini_240911482/lab2$ touch data.txt
4ITB2@debian:~/hridini_240911482/lab2$ grep -E '^..$' data.txt
4ITB2@debian:~/hridini_240911482/lab2$ grep -E '^..$' data.txt
ch
me
un
4ITB2@debian:~/hridini_240911482/lab2$ grep -E '^[A-Z]' data.txt
Apple
ALL CAPS
4ITB2@debian:~/hridini_240911482/lab2$ grep -E '\.$' data.txt
aur lines lo.
pineapple.
4ITB2@debian:~/hridini_240911482/lab2$ grep -E ' +' data.txt
yeh kaun hai
ALL CAPS
aur lines lo.
4ITB2@debian:~/hridini_240911482/lab2$ grep -E '[0-9]' data.txt > digits.txt
```

3. Create file studentInformation.txt using vi editor which contains details in the
following format.
RegistrationNo:Name:Department:Branch:Section:Sub1:Sub2:Sub3
1234:XYZ:ICT:CCE:A:80:60:70 … (add atleast 10 rows)
i) Display the number students(only count) belonging to ICT department.
ii) Replace all occurrences of IT branch with “Information Technology” and save
the output to ITStudents.txt
iii) Display the average marks of student with the given registration number
“1234” (or any specific existing number).



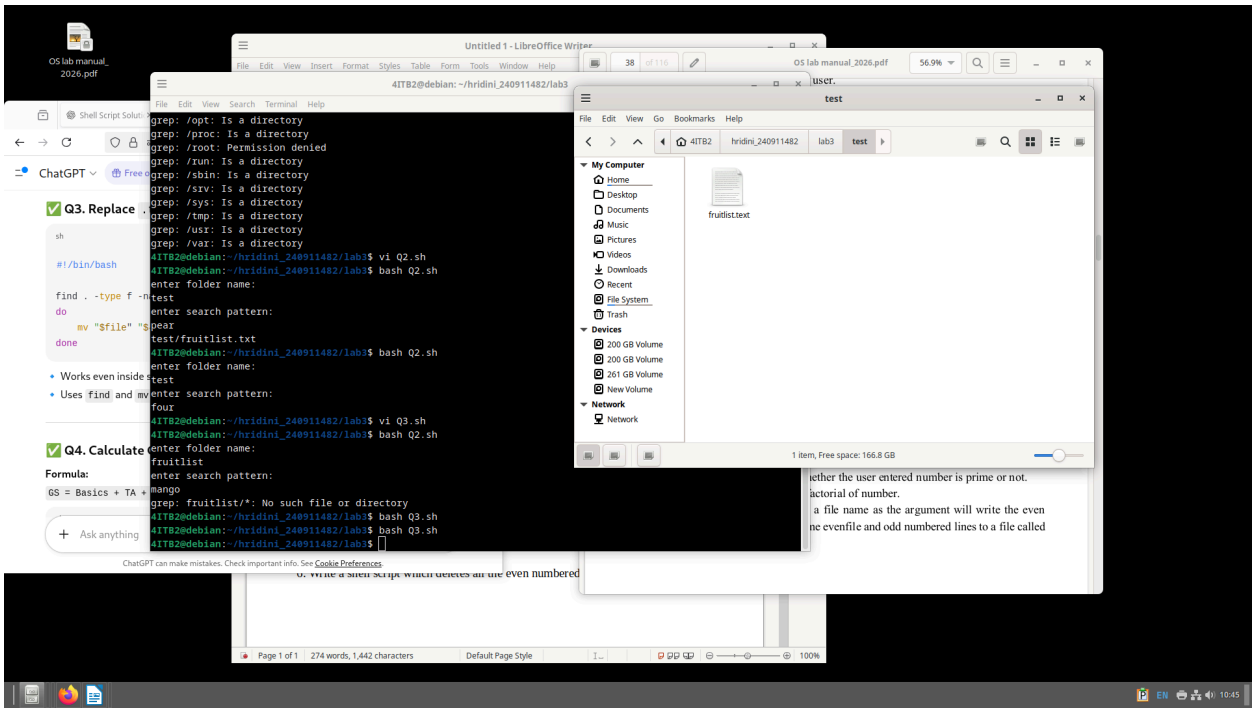

```
4ITB2@debian:~/hridini_240911482/lab2$ grep -E '[0-9]' data.txt > digits.txt
4ITB2@debian:~/hridini_240911482/lab2$ vi studentinfo.txt
4ITB2@debian:~/hridini_240911482/lab2$ grep -c ':ICT:' studentInformation.txt
grep: studentInformation.txt: No such file or directory
4ITB2@debian:~/hridini_240911482/lab2$ sed 's/:IT:/:Information Technology:/g' studentinfo.txt > ITStudents.txt
4ITB2@debian:~/hridini_240911482/lab2$ grep '^1234:' studentinfo.txt | \
awk -F':' '{ avg=($6+$7+$8)/3; print avg }'
70
4ITB2@debian:~/hridini_240911482/lab2$
```

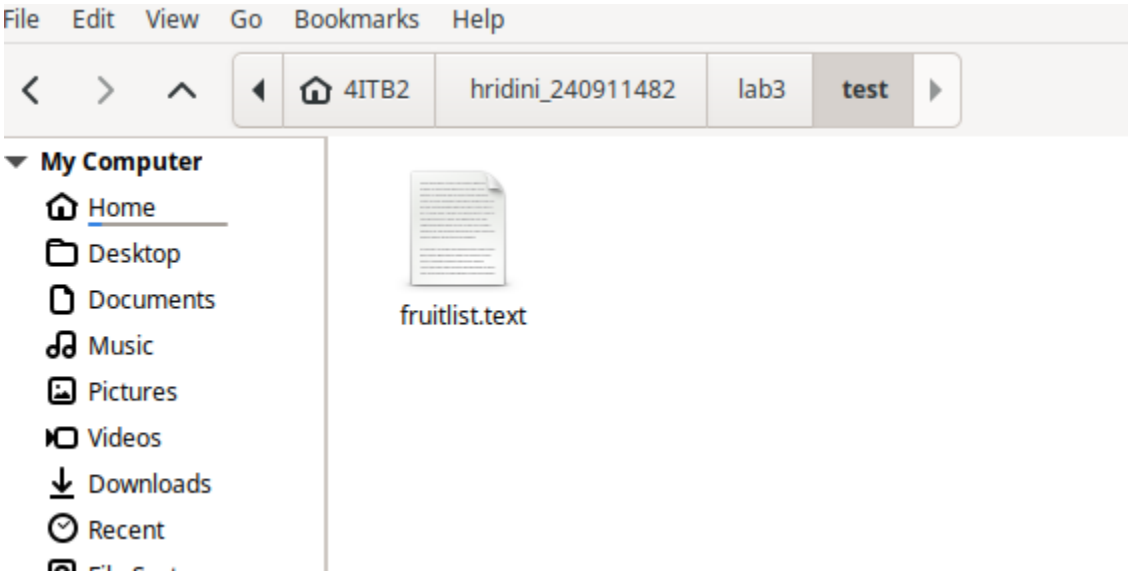
LAB NO: 3

UNIX SHELL PROGRAMMING (SHELL SCRIPTING)

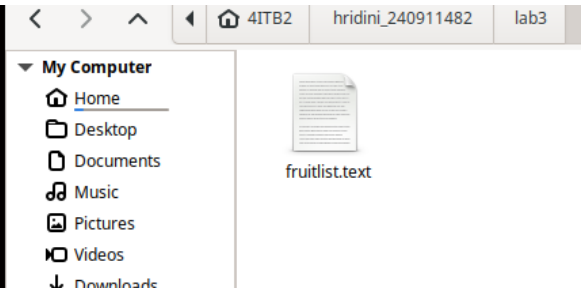
Lab Exercises

1. Write a shell script to find whether a given file is the directory or regular file.

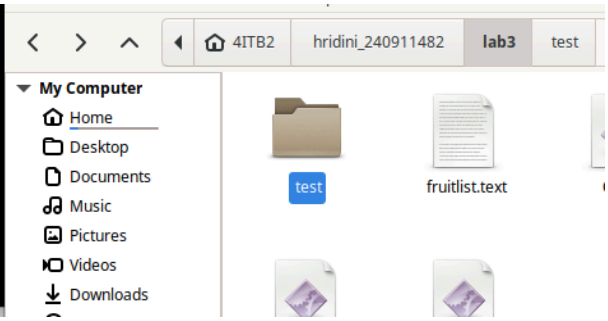


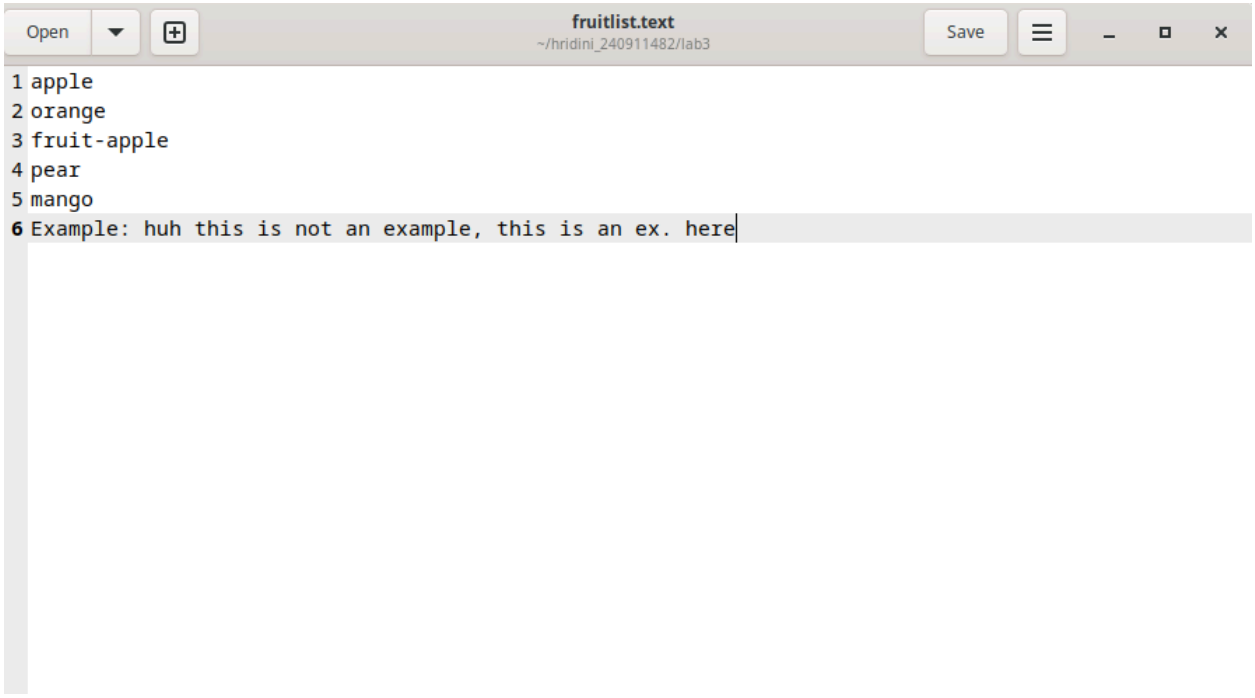


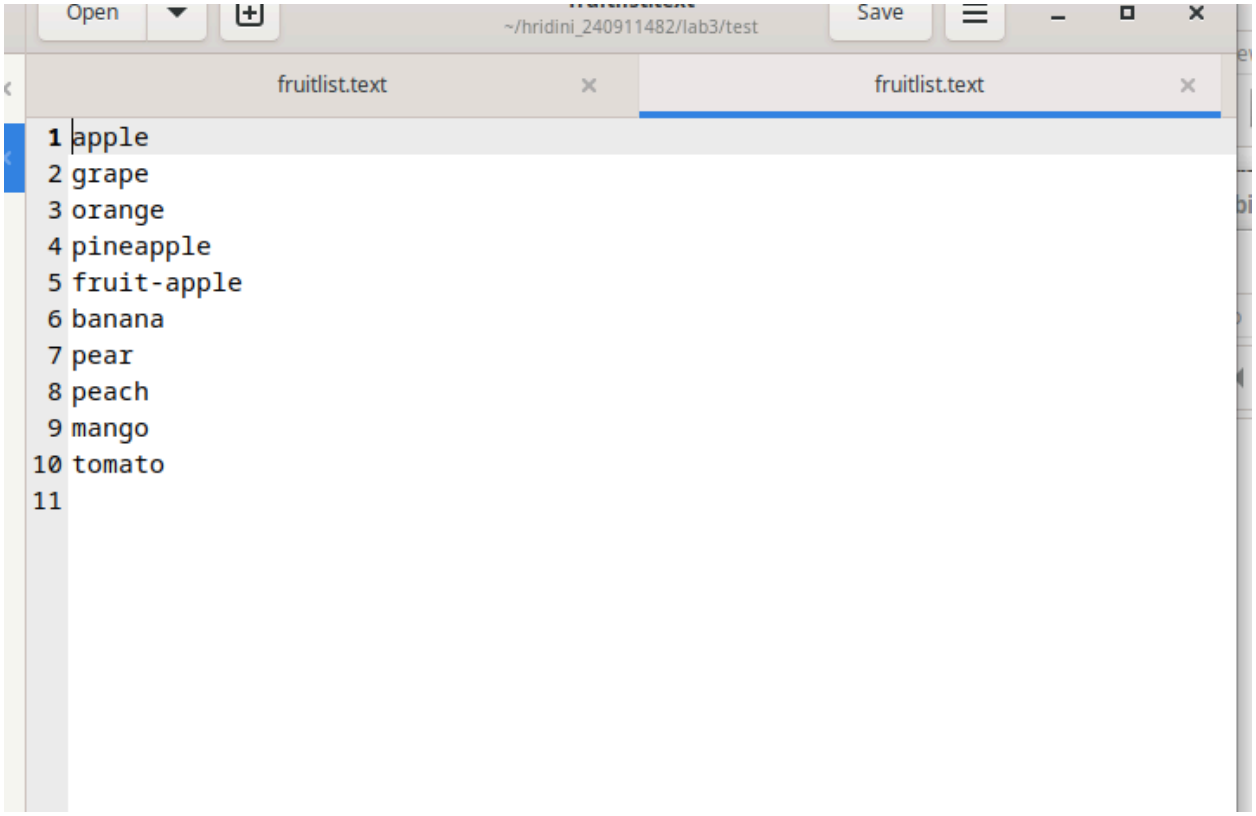
```
ITB2@debian:~/hridini_240911482/lab3$ vi Q3.sh
ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
nter folder name:
ruitlist
nter search pattern:
ango
rep: fruitlist/*: No such file or directory
ITB2@debian:~/hridini_240911482/lab3$ bash Q3.sh
ITB2@debian:~/hridini_240911482/lab3$ bash Q3.sh
```




```
four
4ITB2@debian:~/hridini_240911482/lab3$ vi Q3.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
enter folder name:
fruitlist
enter search pattern:
mango
grep: fruitlist/*: No such file or directory
4ITB2@debian:~/hridini_240911482/lab3$ bash Q3.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q3.sh
4ITB2@debian:~/hridini_240911482/lab3$
```



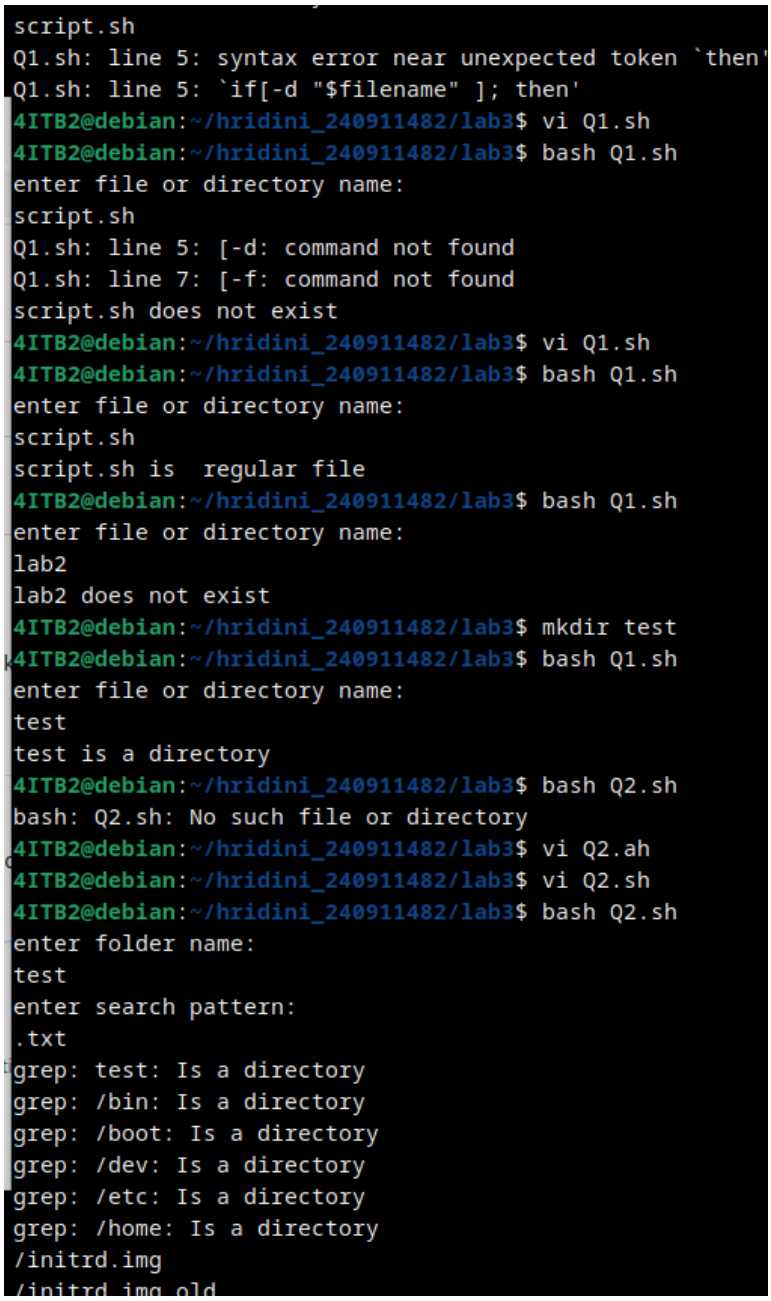



Open
Save
~/hridini_240911482/lab3/test
fruitlist.text
fruitlist.text
1 apple
2 grape
3 orange
4 pineapple
5 fruit-apple
6 banana
7 pear
8 peach
9 mango
10 tomato
11

```
4ITB2@debian:~$ cd hridini_240911482
4ITB2@debian:~/hridini_240911482$ mkdir lab3
4ITB2@debian:~/hridini_240911482$ cd lab3
4ITB2@debian:~/hridini_240911482/lab3$ #!/bin/sh
4ITB2@debian:~/hridini_240911482/lab3$ vi script.sh
4ITB2@debian:~/hridini_240911482/lab3$ ./script.sh
bash: ./script.sh: Permission denied
4ITB2@debian:~/hridini_240911482/lab3$ source script.sh
HRIDINI
4ITB2@debian:~/hridini_240911482/lab3$ echo $value
HRIDINI
4ITB2@debian:~/hridini_240911482/lab3$ result=$(./script.sh
bash: ./script.sh: Permission denied
4ITB2@debian:~/hridini_240911482/lab3$ chmod +x script.sh
4ITB2@debian:~/hridini_240911482/lab3$ result=$(./script.sh
4ITB2@debian:~/hridini_240911482/lab3$ echo $result
HRIDINI
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash script.sh
HRIDINI
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
script.sh
Q1.sh: line 5: syntax error near unexpected token `then'
Q1.sh: line 5: `if[-d "$filename"]; then'
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
script.sh
Q1.sh: line 5: syntax error near unexpected token `then'
Q1.sh: line 5: `if[-d "$filename" ]; then'
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
script.sh
```

2. Write a shell script to list all files (only file names) containing the input pattern
(string) in the folder entered by the user.
3. Write a shell script to replace all files with .txt extension with .text in the current di-
rectory. This has to be done recursively i.e if the current folder contains a folder
“OS” with abc.txt then it has to be changed to abc.text (Hint: use find, mv)
4. Write a shell script to calculate the gross salary. GS=Basics + TA + 10% of Basics.
Floating point calculations has to be performed.
5. Write a program to copy all the files (having file extension input by the user) in the
current folder to the new folder input by the user. ex: user enter .text TEXT then all
files with .text should be moved to TEXT folder. This should be done only at single
level. i.e if the current folder contains a folder name ABC which has .txt files then
these files should not be copied to TEXT.
Write a shell script to modify all occurrences of “ex:” with “Example:” in all the filespresent
31LAB NO: 3
in current folder only if “ex:” occurs at the start of the line or after a period (.). Example:
if a file contains a line: “ex: this is first occurrence so should be re-placed” and “second
ex: should not be replaced as it occurs in the middle of the sen-tence.”6. Write a shell script which deletes all the even

numbered lines in a text file.



```
script.sh
Q1.sh: line 5: syntax error near unexpected token `then'
Q1.sh: line 5: `if[-d "$filename" ]; then'
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
script.sh
Q1.sh: line 5: [-d: command not found
Q1.sh: line 7: [-f: command not found
script.sh does not exist
4ITB2@debian:~/hridini_240911482/lab3$ vi Q1.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
script.sh
script.sh is  regular file
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
lab2
lab2 does not exist
4ITB2@debian:~/hridini_240911482/lab3$ mkdir test
4ITB2@debian:~/hridini_240911482/lab3$ bash Q1.sh
enter file or directory name:
test
test is a directory
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
bash: Q2.sh: No such file or directory
4ITB2@debian:~/hridini_240911482/lab3$ vi Q2.ah
4ITB2@debian:~/hridini_240911482/lab3$ vi Q2.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
enter folder name:
test
enter search pattern:
.txt
grep: test: Is a directory
grep: /bin: Is a directory
grep: /boot: Is a directory
grep: /dev: Is a directory
grep: /etc: Is a directory
grep: /home: Is a directory
/initrd.img
/initrd.img.old
```

```
grep: /sys: Is a directory
grep: /tmp: Is a directory
grep: /usr: Is a directory
grep: /var: Is a directory
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
enter folder name:
test
enter search pattern:
fruit
grep: test: Is a directory
grep: /bin: Is a directory
grep: /boot: Is a directory
grep: /dev: Is a directory
grep: /etc: Is a directory
grep: /home: Is a directory
grep: /lib: Is a directory
grep: /lib32: Is a directory
grep: /lib64: Is a directory
grep: /libx32: Is a directory
grep: /lost+found: Permission denied
grep: /media: Is a directory
grep: /mnt: Is a directory
grep: /opt: Is a directory
grep: /proc: Is a directory
grep: /root: Permission denied
grep: /run: Is a directory
grep: /sbin: Is a directory
grep: /srv: Is a directory
grep: /sys: Is a directory
grep: /tmp: Is a directory
grep: /usr: Is a directory
grep: /var: Is a directory
4ITB2@debian:~/hridini_240911482/lab3$ vi Q2.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
enter folder name:
test
enter search pattern:
pear
test/fruitlist.txt
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
enter folder name:
test
enter search pattern:
four
```

```
test
enter search pattern:
four
4ITB2@debian:~/hridini_240911482/lab3$ vi Q3.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q2.sh
enter folder name:
fruitlist
enter search pattern:
mango
grep: fruitlist/*: No such file or directory
4ITB2@debian:~/hridini_240911482/lab3$ bash Q3.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q3.sh
4ITB2@debian:~/hridini_240911482/lab3$ vi Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
Q4.sh: line 3: syntax error near unexpected token `hit'
Q4.sh: line 3: `$bc(hit enter)5+2'
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
Q4.sh: line 3: syntax error near unexpected token `hit'
Q4.sh: line 3: `$bc(hit enter)5+2'
4ITB2@debian:~/hridini_240911482/lab3$ vi Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ vi Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
7
4ITB2@debian:~/hridini_240911482/lab3$ vi Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
7
4ITB2@debian:~/hridini_240911482/lab3$ vi Q4.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q4.sh
Enter Basic Salary:
5000
Enter TA:
600
Gross Salary is 6100.00
4ITB2@debian:~/hridini_240911482/lab3$ vi Q5.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q5.sh
Enter file extension (example: .text):
.text
Enter destination folder name:
test
Files copied successfully
4ITB2@debian:~/hridini_240911482/lab3$ bash Q5.sh
Enter file extension (example: .text):
.sh
```

```
4ITB2@debian:~/hridini_240911482/lab3$ bash Q5.sh
Enter file extension (example: .text):
.sh
Enter destination folder name:
test
Files copied successfully
4ITB2@debian:~/hridini_240911482/lab3$ bash Q5b.sh
bash: Q5b.sh: No such file or directory
4ITB2@debian:~/hridini_240911482/lab3$ vi Q5b.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q5b.sh
4ITB2@debian:~/hridini_240911482/lab3$ vi Q6.sh
4ITB2@debian:~/hridini_240911482/lab3$ bash Q6.sh
Enter file name:
fruitlist
sed: can't read fruitlist: No such file or directory
4ITB2@debian:~/hridini_240911482/lab3$ bash Q6.sh
Enter file name:
fruitlist.text
```

LAB NO: 4

PROCESS AND THREAD MANAGEMENT

Lab Exercises

1. Write a C program to create a child process. Display different messages in parent process and child process. Display PID and PPID of both parent and child process. Block parent process until child completes using wait system call.

2. Write a C program to load the binary executable of the previous program in a child process using exec system call.

3. Create a zombie (defunct) child process (a child with exit() call, but no corresponding wait() in the sleeping parent) and allow the init process to adopt it (after parent terminates). Run the process as background process and run “ps” Command.

ex1.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }
    else if (pid == 0) {   // Child
        printf("Child Process\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
        exit(0);
    }
    else {                 // Parent
        wait(NULL);
        printf("Parent Process\n");
        printf("PID  = %d\n", getpid());
        printf("PPID = %d\n", getppid());
    }
    return 0;
}
```

ex2.c

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        execl("./ex1", "ex1", NULL);

        // Runs only if exec fails
        perror("exec failed");
        exit(1);
    }
    else {
        // Parent process
        wait(NULL);
        printf("Parent: ex1 execution completed\n");
    }

    return 0;
}
```

ex3.c

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {    // Child
        printf("Child exiting\n");
        exit(0);
    }
    else {             // Parent
        sleep(20);     // Parent sleeps, no wait()
        printf("Parent exiting\n");
    }
    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week4$ gcc example2.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
usage: ./a.out command [args...]
4ITB2@debian:~/hridini_240911482/week4$ vi ex1.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex1.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Child Process
PID  = 379259
PPID = 379258
Parent Process
PID  = 379258
PPID = 17658
4ITB2@debian:~/hridini_240911482/week4$ gcc ex1.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Child Process
PID  = 379269
PPID = 379268
Parent Process
PID  = 379268
PPID = 17658
4ITB2@debian:~/hridini_240911482/week4$ gcc ex1.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Child Process
PID  = 380564
PPID = 380563
Parent Process
PID  = 380563
PPID = 17658
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Child Process
PID  = 381854
PPID = 381853
Parent Process
PID  = 381853
PPID = 17658
4ITB2@debian:~/hridini_240911482/week4$ vi ex2.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex2.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
exec failed: No such file or directory
```

```
4ITB2@debian:~/hridini_240911482/week4$ ls
a.out  ex1.c  ex2.c  example2.c  example.c  fork1.c  hello.c  wait1.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex1.c -o ex1
4ITB2@debian:~/hridini_240911482/week4$ ls
a.out  ex1  ex1.c  ex2.c  example2.c  example.c  fork1.c  hello.c  wait1.c
4ITB2@debian:~/hridini_240911482/week4$ vi ex2.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex2.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Child Process
PID  = 403793
PPID = 403792
Parent Process
PID  = 403792
PPID = 403791
Parent: ex1 execution completed
4ITB2@debian:~/hridini_240911482/week4$ vi ex3.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex2.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out &
ps
[1] 419271
Child Process
PID  = 419274
PPID = 419273
Parent Process
PID  = 419273
PPID = 419271
Parent: ex1 execution completed
    PID TTY          TIME CMD
  17658 pts/0    00:00:00 bash
 419272 pts/0    00:00:00 ps
[1]+  Done                    ./a.out
4ITB2@debian:~/hridini_240911482/week4$ ./a.out &
[1] 419275
4ITB2@debian:~/hridini_240911482/week4$ Child Process
PID  = 419277
PPID = 419276
Parent Process
PID  = 419276
PPID = 419275
```

```
4ITB2@debian:~/hridini_240911482/week4$ Child Process
PID  = 419277
PPID = 419276
Parent Process
PID  = 419276
PPID = 419275
Parent: ex1 execution completed
ps
    PID TTY          TIME CMD
  17658 pts/0    00:00:00 bash
 419278 pts/0    00:00:00 ps
[1]+  Done                    ./a.out
4ITB2@debian:~/hridini_240911482/week4$ vi ex4.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex4.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex3.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out & ps
[1] 425741
Child exiting
    PID TTY          TIME CMD
  17658 pts/0    00:00:00 bash
 425741 pts/0    00:00:00 a.out
 425742 pts/0    00:00:00 ps
 425743 pts/0    00:00:00 a.out
4ITB2@debian:~/hridini_240911482/week4$ ./a.out &
[2] 427032
4ITB2@debian:~/hridini_240911482/week4$ Child exiting
ps
    PID TTY          TIME CMD
  17658 pts/0    00:00:00 bash
 425741 pts/0    00:00:00 a.out
 425743 pts/0    00:00:00 a.out
 427032 pts/0    00:00:00 a.out
 427033 pts/0    00:00:00 a.out
 427034 pts/0    00:00:00 ps
4ITB2@debian:~/hridini_240911482/week4$ Parent exiting

[1]-  Done                    ./a.out
4ITB2@debian:~/hridini_240911482/week4$ Parent exiting
./a.out & ps
```

```
[3] 428325
Child exiting
    PID TTY          TIME CMD
  17658 pts/0    00:00:00 bash
 428325 pts/0    00:00:00 a.out
 428326 pts/0    00:00:00 ps
 428327 pts/0    00:00:00 a.out
[2]   Done                    ./a.out
```

4. Write a multithreaded program that performs different sorting algorithms. The program should work as follows: the user enters on the command line the number of elements to sort and the elements themselves. The program then creates separate threads, each using a different sorting algorithm. Each thread sorts the array using its corresponding algorithm and displays the time taken to produce the result. The main thread waits for all threads to finish and then displays the final sorted array.

ex4.c

```c
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
#include <time.h>

int n;
int arr[100];

void* bubble_sort(void* arg) {
    clock_t start = clock();
    int temp[100];
    for (int i = 0; i < n; i++) temp[i] = arr[i];

    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (temp[j] > temp[j+1]) {
                int t = temp[j];
                temp[j] = temp[j+1];
                temp[j+1] = t;
            }

    clock_t end = clock();
    printf("Bubble Sort Time: %f\n", (double)(end-start)/CLOCKS_PER_SEC);
    pthread_exit(NULL);
}

void* selection_sort(void* arg) {
    clock_t start = clock();
    int temp[100];
    for (int i = 0; i < n; i++) temp[i] = arr[i];

    for (int i = 0; i < n-1; i++) {
        int min = i;
        for (int j = i+1; j < n; j++)
            if (temp[j] < temp[min]) min = j;
        int t = temp[i];
        temp[i] = temp[min];
        temp[min] = t;
    }
```

```c

    clock_t end = clock();
    printf("Selection Sort Time: %f\n", (double)(end-start)/CLOCKS_PER_SEC);
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    pthread_create(&t1, NULL, bubble_sort, NULL);
    pthread_create(&t2, NULL, selection_sort, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    // Final sorted array (bubble sort)
    for (int i = 0; i < n-1; i++)
        for (int j = 0; j < n-i-1; j++)
            if (arr[j] > arr[j+1]) {
                int t = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = t;
            }

    printf("Final Sorted Array:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);

    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week4$ gcc ex4.c
4ITB2@debian:~/hridini_240911482/week4$ gcc Parent exiting

gcc: fatal error: no input files
compilation terminated.
[3]+  Done                    ./a.out
4ITB2@debian:~/hridini_240911482/week4$ gcc ex4.c
4ITB2@debian:~/hridini_240911482/week4$ gcc ex4.c -lpthread
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Enter number of elements: 4
Enter elements:
50
21
7
19
Bubble Sort Time: 0.000002
Selection Sort Time: 0.000003
Final Sorted Array:
7 19 21 50 4ITB2@debian:~/hridini_240911482/week4$ vi ex5.c
```

5. Write multithreaded program that generates the Fibonacci series. The program should work as follows: The user will enter on the command line the number of Fibonacci numbers that the program is to generate. The program then will create a separate thread that will generate the Fibonacci numbers, placing the sequence in data that is shared by the threads (an array is probably the most convenient data structure). When the thread finishes execution the parent will output the sequence generated by the child thread. Because the parent thread cannot begin outputting the Fibonacci sequence until the child thread finishes, his will require having the parent thread wait for the child thread to finish.

ex5.c

```c
#include <stdio.h>
#include <pthread.h>

int fib[100];
int n;

void* generate_fib(void* arg) {
    fib[0] = 0;
    fib[1] = 1;

    for (int i = 2; i < n; i++)
        fib[i] = fib[i-1] + fib[i-2];

    pthread_exit(NULL);
}

int main() {
    pthread_t thread;

    printf("Enter number of Fibonacci terms: ");
    scanf("%d", &n);

    pthread_create(&thread, NULL, generate_fib, NULL);
    pthread_join(thread, NULL);

    printf("Fibonacci Series:\n");
    for (int i = 0; i < n; i++)
        printf("%d ", fib[i]);

    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week4$ gcc ex5.c
4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Enter number of Fibonacci terms: 5
Fibonacci Series:
0 1 1 2 3 4ITB2@debian:~/hridini_240911482/week4$ ./a.out
Enter number of Fibonacci terms: 19
Fibonacci Series:
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597 2584 4ITB2@debian:~/hridini_24091
```

LAB NO:5

CPU SCHEDULING ALGORITHMS

Lab Exercise

1. Consider the following set of processes, with length of the CPU burst given in
milliseconds:

Process	Arrival time	Burst time	*Priority*
P1	0	60	3
P2	3	30	2
P3	4	40	1
P4	9	10	4

Write a menu driven C/C++ program to simulate the following CPU scheduling algorithms.
Display Gantt chart showing the order of execution of each process. Compute waiting time and
turnaround time for each process. Hence, compute average waiting time and average turnaround
time.
(i) FCFS (ii) SRTF (iii) Round-Robin (quantum = 10) iv) non-preemptive priority (higher
the number higher the priority)

Q1.c

```c
#include <stdio.h>

#define MAX 20

void fcfs(int n, int at[], int bt[]);
void srtf(int n, int at[], int bt[]);
void roundRobin(int n, int at[], int bt[], int quantum);
void priorityNP(int n, int at[], int bt[], int pr[]);

void fcfs(int n, int at[], int bt[]) {
    int ct[MAX], wt[MAX], tat[MAX];
    int time = 0;

    printf("\nGantt Chart:\n");

    for(int i = 0; i < n; i++) {

        if(time < at[i])
            time = at[i];

        time += bt[i];
        ct[i] = time;

        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        printf("P%d ", i+1);
    }

    float avg_wt = 0, avg_tat = 0;

    printf("\n\nProcess\tWT\tTAT\n");
    for(int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\n", i+1, wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("Average WT = %.2f\n", avg_wt/n);
    printf("Average TAT = %.2f\n", avg_tat/n);
}
```

```c
void srtf(int n, int at[], int bt[]) {
    int rt[MAX], ct[MAX], wt[MAX], tat[MAX];
    int completed = 0, time = 0, min, shortest;

    for(int i = 0; i < n; i++)
        rt[i] = bt[i];
    printf("\nGantt Chart:\n");
    while(completed != n) {

        min = 9999;
        shortest = -1;

        for(int i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0 && rt[i] < min) {
                min = rt[i];
                shortest = i;}}
        if(shortest == -1) {
            time++;
            continue;}

        rt[shortest]--;
        printf("P%d ", shortest+1);
        time++;

        if(rt[shortest] == 0) {
            completed++;
            ct[shortest] = time;
        }}
    float avg_wt = 0, avg_tat = 0;
    printf("\n\nProcess\tWT\tTAT\n");
    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        printf("P%d\t%d\t%d\n", i+1, wt[i], tat[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("Average WT = %.2f\n", avg_wt/n);
    printf("Average TAT = %.2f\n", avg_tat/n);
}
```

```c
void roundRobin(int n, int at[], int bt[], int quantum) {
    int rt[MAX], wt[MAX], tat[MAX], ct[MAX];
    int time = 0, completed = 0;

    for(int i = 0; i < n; i++)
        rt[i] = bt[i];

    printf("\nGantt Chart:\n");

    while(completed != n) {

        for(int i = 0; i < n; i++) {

            if(rt[i] > 0 && at[i] <= time) {

                if(rt[i] > quantum) {
                    time += quantum;
                    rt[i] -= quantum;
                    printf("P%d ", i+1);
                }
                else {
                    time += rt[i];
                    printf("P%d ", i+1);
                    rt[i] = 0;
                    ct[i] = time;
                    completed++;
                }}}}

    float avg_wt = 0, avg_tat = 0;
    printf("\n\nProcess\tWT\tTAT\n");

    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        printf("P%d\t%d\t%d\n", i+1, wt[i], tat[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];}
    printf("Average WT = %.2f\n", avg_wt/n);
    printf("Average TAT = %.2f\n", avg_tat/n);
}
```

```c
void priorityNP(int n, int at[], int bt[], int pr[]) {
    int completed = 0, time = 0;
    int visited[MAX] = {0};
    int ct[MAX], wt[MAX], tat[MAX];

    printf("\nGantt Chart:\n");

    while(completed != n) {

        int idx = -1;
        int highest = -1;

        for(int i = 0; i < n; i++) {
            if(at[i] <= time && visited[i] == 0) {
                if(pr[i] > highest) {
                    highest = pr[i];
                    idx = i;
                } } }

        if(idx == -1) {
            time++;
            continue;
        }

        time += bt[idx];
        ct[idx] = time;
        visited[idx] = 1;
        completed++;

        printf("P%d ", idx+1);
    }
    float avg_wt = 0, avg_tat = 0;
    printf("\n\nProcess\tWT\tTAT\n");

    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];

        printf("P%d\t%d\t%d\n", i+1, wt[i], tat[i]);

        avg_wt += wt[i];
        avg_tat += tat[i];}
    printf("Average WT = %.2f\n", avg_wt/n);
    printf("Average TAT = %.2f\n", avg_tat/n);
}
```

```c
int main() {
    int n, choice;
    int at[MAX], bt[MAX], pr[MAX];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("Process %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority: ");
        scanf("%d", &pr[i]);
    }

    do {
        printf("\n1.FCFS\n2.SRTF\n3.Round Robin\n4.Pri
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch(choice) {
            case 1: fcfs(n, at, bt); break;
            case 2: srtf(n, at, bt); break;
            case 3: roundRobin(n, at, bt, 10); break;
            case 4: priorityNP(n, at, bt, pr); break;
        }
    } while(choice != 5);

    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/lab5$ gcc Q1.c
4ITB2@debian:~/hridini_240911482/lab5$ ./a.out
Enter number of processes: 4
Process 1
Arrival Time: 0
Burst Time: 60
Priority: 3
Process 2
Arrival Time: 3
Burst Time: 30
Priority: 2
Process 3
Arrival Time: 4
Burst Time: 40
Priority: 1
Process 4
Arrival Time: 9
Burst Time: 10
Priority: 4

1.FCFS
2.SRTF
3.Round Robin
4.Priority (Non-preemptive)
5.Exit
Enter choice: 1

Gantt Chart:
P1 P2 P3 P4

Process WT      TAT
P1      0       60
P2      57      87
P3      86      126
P4      121     131
Average WT = 66.00
Average TAT = 101.00

1.FCFS
2.SRTF
3.Round Robin
4.Priority (Non-preemptive)
5.Exit
```

```
Enter choice: 2

Gantt Chart:
P1 P1 P1 P2 P2 P2 P2 P2 P2 P4 P4 P4 P4 P4 P4 P4 P4 P4 P2 P2 P2 P2 P2 P2 P2 P2 P2 P
2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3
 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P3 P1 P1
P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P
1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1

Process WT      TAT
P1      80      140
P2      10      40
P3      39      79
P4      0       10
Average WT = 32.25
Average TAT = 67.25

1.FCFS
2.SRTF
3.Round Robin
4.Priority (Non-preemptive)
5.Exit
Enter choice: 3

Gantt Chart:
P1 P2 P3 P4 P1 P2 P3 P1 P2 P3 P1 P3 P1 P1

Process WT      TAT
P1      80      140
P2      57      87
P3      76      116
P4      21      31
Average WT = 58.50
Average TAT = 93.50

1.FCFS
2.SRTF
3.Round Robin
4.Priority (Non-preemptive)
5.Exit
```

```
Enter choice: 4

Gantt Chart:
P1 P4 P2 P3

Process WT      TAT
P1      0       60
P2      67      97
P3      96      136
P4      51      61
Average WT = 53.50
Average TAT = 88.50

1.FCFS
2.SRTF
3.Round Robin
4.Priority (Non-preemptive)
5.Exit
Enter choice: 5
```

2. Write a C program to simulate multi-level feedback queue scheduling algorithm.

Q2.c

```c
#include <stdio.h>
#define MAX 20
int main() {
    int n;
    int at[MAX], bt[MAX], rt[MAX];
    int ct[MAX], wt[MAX], tat[MAX];
    int time = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        rt[i] = bt[i];   // copy burst time
    }
    printf("\nGantt Chart:\n");
    // -------- Q0 (Quantum = 8) --------
    for(int i = 0; i < n; i++) {
        if(rt[i] > 0) {
            if(rt[i] > 8) {
                time += 8;
                rt[i] -= 8;
                printf("P%d ", i+1);
            } else {
                time += rt[i];
                printf("P%d ", i+1);
                rt[i] = 0;
                ct[i] = time;
            }}}
```

```
34
35      // -------- Q1 (Quantum = 16) --------
36      for(int i = 0; i < n; i++) {
37          if(rt[i] > 0) {
38              if(rt[i] > 16) {
39                  time += 16;
40                  rt[i] -= 16;
41                  printf("P%d ", i+1);
42              } else {
43                  time += rt[i];
44                  printf("P%d ", i+1);
45                  rt[i] = 0;
46                  ct[i] = time;
47              }}}
48
49      // -------- Q2 (FCFS) --------
50      for(int i = 0; i < n; i++) {
51          if(rt[i] > 0) {
52              time += rt[i];
53              printf("P%d ", i+1);
54              rt[i] = 0;
55              ct[i] = time;
56          }}
57      float avg_wt = 0, avg_tat = 0;
58      printf("\n\nProcess\tWT\tTAT\n");
59      for(int i = 0; i < n; i++) {
60          tat[i] = ct[i] - at[i];
61          wt[i] = tat[i] - bt[i];
62          printf("P%d\t%d\t%d\n", i+1, wt[i], tat[i]);
63          avg_wt += wt[i];
64          avg_tat += tat[i];}
65
66      printf("\nAverage Waiting Time = %.2f", avg_wt/n);
67      printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);
68
69      return 0;
70  }
```

```
4ITB2@debian:~/hridini_240911482/lab5$ vi Q2.c
4ITB2@debian:~/hridini_240911482/lab5$ gcc Q2.c
/usr/bin/ld: /usr/lib/gcc/x86_64-linux-gnu/12/../../../x86_64-linux-gnu/Scrt1.o: in f
unction `_start':
(.text+0x17): undefined reference to `main'
collect2: error: ld returned 1 exit status
4ITB2@debian:~/hridini_240911482/lab5$ vi Q2.c
4ITB2@debian:~/hridini_240911482/lab5$ vi Q2.c
4ITB2@debian:~/hridini_240911482/lab5$ gcc Q2.c
4ITB2@debian:~/hridini_240911482/lab5$ ./a.out
Enter number of processes: 2

Process 1
Arrival Time: 0
Burst Time: 3

Process 2
Arrival Time: 1
Burst Time: 2

Gantt Chart:
P1 P2

Process WT      TAT
P1      0       3
P2      2       4

Average Waiting Time = 1.00
Average Turnaround Time = 3.50
```

3. Write a C program to simulate Earliest-Deadline-First scheduling for realtime systems.

Q3.c

```c
#include <stdio.h>
#define MAX 20

int main() {
    int n;
    int at[MAX], bt[MAX], deadline[MAX];
    int rt[MAX], ct[MAX], wt[MAX], tat[MAX];
    int time = 0, completed = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Deadline: ");
        scanf("%d", &deadline[i]);

        rt[i] = bt[i];}
    printf("\nGantt Chart:\n");
    while(completed != n) {

        int idx = -1;
        int earliest = 9999;

        for(int i = 0; i < n; i++) {
            if(at[i] <= time && rt[i] > 0) {
                if(deadline[i] < earliest) {
                    earliest = deadline[i];
                    idx = i;
                }}}

        if(idx == -1) {
            time++;
            continue;}
        rt[idx]--;
        printf("P%d ", idx+1);
        time++;

        if(rt[idx] == 0) {
            ct[idx] = time;
            completed++;
        }}
```

```c
    float avg_wt = 0, avg_tat = 0;
    printf("\n\nProcess\tWT\tTAT\n");
    for(int i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
        printf("P%d\t%d\t%d\n", i+1, wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];}

    printf("\nAverage Waiting Time = %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", avg_tat/n);

    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/lab5$ vi Q3.c
4ITB2@debian:~/hridini_240911482/lab5$ gcc Q3.c
4ITB2@debian:~/hridini_240911482/lab5$ ./a.out
Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 60
Deadline:

^[

Process 2
Arrival Time: Burst Time: Deadline:
Process 3
Arrival Time: Burst Time: Deadline:
Process 4
Arrival Time: Burst Time: Deadline:
Gantt Chart:

^[^[^[^[
^C
4ITB2@debian:~/hridini_240911482/lab5$ ./a.out
Enter number of processes: 2

Process 1
Arrival Time: 50
Burst Time: 25
Deadline: 50

Process 2
Arrival Time: 80
Burst Time: 35
Deadline: 80

Gantt Chart:
P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P1 P2 P2 P2 P
2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2 P2
 P2 P2 P2

Process WT      TAT
P1      0       25
P2      0       35

Average Waiting Time = 0.00
Average Turnaround Time = 30.00
```

LAB NO.: 6

INTERPROCESS COMMUNICATION

Lab Exercises:

1. Process A wants to send a number to Process B. Once received, Process B has to check
whether the number is palindrome or not. Write a C program to implement this interprocess
communication using message queue.

sender.c receiver.c

```c
// sender.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>

struct msg_st {
    long int msg_type;
    char text[100];
};
int main() {
    int msgid;
    struct msg_st data;
    msgid = msgget((key_t)1234, 0666 | IPC_CREAT);
    printf("Enter a number: ");
    scanf("%s", data.text);
    data.msg_type = 1;
    msgsnd(msgid, (void *)&data, sizeof(data.text), 0);
    printf("Number sent to Process B\n");
    return 0;
}
```

sender.c receiver.c

```c
// receiver.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <string.h>
struct msg_st {
    long int msg_type;
    char text[100];
};
int isPalindrome(char num[]) {
    int i, len = strlen(num);
    for(i = 0; i < len/2; i++) {
        if(num[i] != num[len - i - 1])
            return 0;}
    return 1;
}
int main() {
    int msgid;
    struct msg_st data;
    msgid = msgget((key_t)1234, 0666 | IPC_CREAT);
    msgrcv(msgid, (void *)&data, sizeof(data.text), 0, 0);
    printf("Received number: %s\n", data.text);
    if(isPalindrome(data.text))
        printf("It is Palindrome\n");
    else
        printf("Not a Palindrome\n");
    msgctl(msgid, IPC_RMID, 0);
    return 0;
}
```

2. Write a producer and consumer program in C using FIFO queue. The producer should write
a set of 4 integers into the FIFO queue and the consumer should display the 4 integers.

producer.c consumer.c

```c
// producer_fifo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>

#define FIFO_NAME "myfifo"

int main() {
    int fd;
    int numbers[4];

    mkfifo(FIFO_NAME, 0666);

    fd = open(FIFO_NAME, O_WRONLY);

    printf("Enter 4 integers:\n");
    for(int i=0;i<4;i++)
        scanf("%d",&numbers[i]);

    write(fd, numbers, sizeof(numbers));

    close(fd);
    return 0;
}
```

producer.c consumer.c

```c
// consumer_fifo.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>

#define FIFO_NAME "myfifo"

int main() {
    int fd;
    int numbers[4];

    fd = open(FIFO_NAME, O_RDONLY);

    read(fd, numbers, sizeof(numbers));

    printf("Received numbers:\n");
    for(int i=0;i<4;i++)
        printf("%d ", numbers[i]);

    printf("\n");

    close(fd);
    unlink(FIFO_NAME);

    return 0;}
```

3. Implement a parent process, which sends an English alphabet to child process using shared
memory. Child process responds back with next English alphabet to the parent. Parent
displays the reply from the Child.

parent_child_alp_shm.c

```c
// parent_child_shm.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int shmid;
    char *shared_mem;

    shmid = shmget(1234, 10, 0666 | IPC_CREAT);
    shared_mem = (char*) shmat(shmid, NULL, 0);
    printf("Enter an alphabet: ");
    scanf(" %c", shared_mem);
    if(fork() == 0) {
        // Child
        *shared_mem = *shared_mem + 1;
        exit(0);}
    else {
        wait(NULL);
        printf("Next alphabet from child: %c\n", *shared_mem);}
    shmdt(shared_mem);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

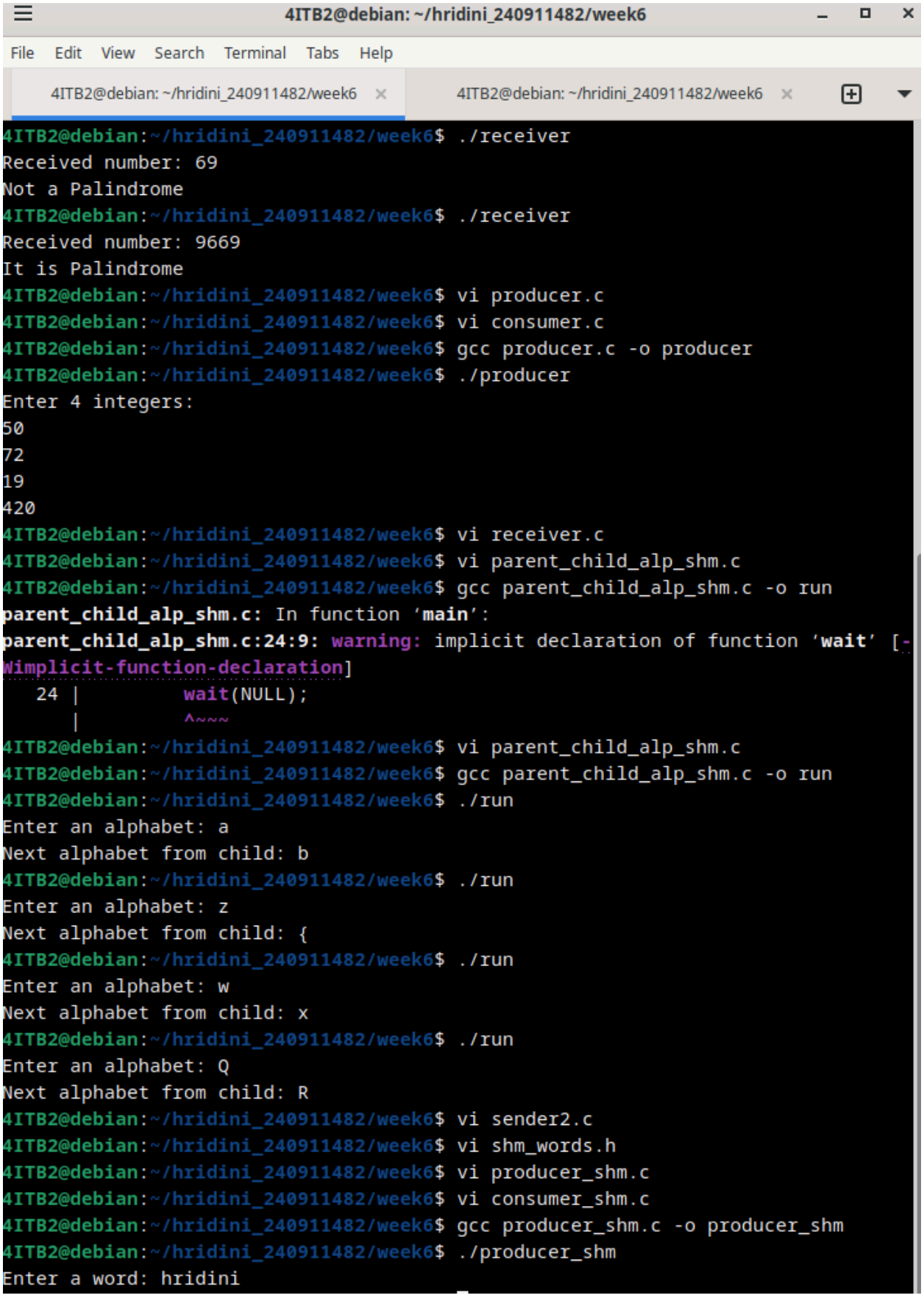
4. Write a producer-consumer program in C in which producer writes a set of words into shared
memory and then consumer reads the set of words from the shared memory. The shared
memory need to be detached and deleted after use.

consumer_shm.c producer_shm.c

```c
// consumer_shm.c
#include <stdio.h>
#include <stdlib.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include "shm_words.h"

int main() {
    int shmid;
    struct shared_data *shared;

    shmid = shmget(1234, sizeof(struct shared_data), 0666 | IPC_CREAT);
    shared = shmat(shmid, NULL, 0);
    while(shared->written != 1);
    printf("Received word: %s\n", shared->text);
    shmdt(shared);
    shmctl(shmid, IPC_RMID, NULL);
    return 0;
}
```

consumer_shm.c producer_shm.c

```c
// producer_shm.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include "shm_words.h"

int main() {
    int shmid;
    struct shared_data *shared;

    shmid = shmget(1234, sizeof(struct shared_data), 0666 | IPC_CREAT);
    shared = shmat(shmid, NULL, 0);
    printf("Enter a word: ");
    scanf("%s", shared->text);
    shared->written = 1;
    shmdt(shared);
    return 0;
}
```


4ITB2@debian: ~/hridini_240911482/week6
File Edit View Search Terminal Tabs Help
4ITB2@debian: ~/hridini_240911482/week6
4ITB2@debian: ~/hridini_240911482/week6
4ITB2@debian:~/hridini_240911482/week6$ ./receiver
Received number: 69
Not a Palindrome
4ITB2@debian:~/hridini_240911482/week6$ ./receiver
Received number: 9669
It is Palindrome
4ITB2@debian:~/hridini_240911482/week6$ vi producer.c
4ITB2@debian:~/hridini_240911482/week6$ vi consumer.c
4ITB2@debian:~/hridini_240911482/week6$ gcc producer.c -o producer
4ITB2@debian:~/hridini_240911482/week6$ ./producer
Enter 4 integers:
50
72
19
420
4ITB2@debian:~/hridini_240911482/week6$ vi receiver.c
4ITB2@debian:~/hridini_240911482/week6$ vi parent_child_alp_shm.c
4ITB2@debian:~/hridini_240911482/week6$ gcc parent_child_alp_shm.c -o run
parent_child_alp_shm.c: In function 'main':
parent_child_alp_shm.c:24:9: warning: implicit declaration of function 'wait' [-
Wimplicit-function-declaration]
24 | wait(NULL);
| ^~~~
4ITB2@debian:~/hridini_240911482/week6$ vi parent_child_alp_shm.c
4ITB2@debian:~/hridini_240911482/week6$ gcc parent_child_alp_shm.c -o run
4ITB2@debian:~/hridini_240911482/week6$ ./run
Enter an alphabet: a
Next alphabet from child: b
4ITB2@debian:~/hridini_240911482/week6$ ./run
Enter an alphabet: z
Next alphabet from child: {
4ITB2@debian:~/hridini_240911482/week6$ ./run
Enter an alphabet: w
Next alphabet from child: x
4ITB2@debian:~/hridini_240911482/week6$ ./run
Enter an alphabet: Q
Next alphabet from child: R
4ITB2@debian:~/hridini_240911482/week6$ vi sender2.c
4ITB2@debian:~/hridini_240911482/week6$ vi shm_words.h
4ITB2@debian:~/hridini_240911482/week6$ vi producer_shm.c
4ITB2@debian:~/hridini_240911482/week6$ vi consumer_shm.c
4ITB2@debian:~/hridini_240911482/week6$ gcc producer_shm.c -o producer_shm
4ITB2@debian:~/hridini_240911482/week6$ ./producer_shm
Enter a word: hridini

```
4ITB2@debian:~/hridini_240911482/week6$ ^[[200~gcc sender.c -o sender
bash: $'\E[200~gcc': command not found
4ITB2@debian:~/hridini_240911482/week6$ ~gcc sender.c -o sender
bash: ~gcc: command not found
4ITB2@debian:~/hridini_240911482/week6$ gcc sender.c -o sender
4ITB2@debian:~/hridini_240911482/week6$ ./sender
Enter a number: 69
Number sent to Process B
4ITB2@debian:~/hridini_240911482/week6$ ./sender
Enter a number: 9669
Number sent to Process B
4ITB2@debian:~/hridini_240911482/week6$ gcc consumer.c -o consumer
4ITB2@debian:~/hridini_240911482/week6$ ./consumer
Received numbers:
0 0 792493936 32528
4ITB2@debian:~/hridini_240911482/week6$ ./consumer
Received numbers:
50 72 19 420
4ITB2@debian:~/hridini_240911482/week6$ gcc consumer_shm.c -o consumer_shm.c
gcc: fatal error: input file 'consumer_shm.c' is the same as output file
compilation terminated.
4ITB2@debian:~/hridini_240911482/week6$ gcc consumer_shm.c -o consumer_shm
4ITB2@debian:~/hridini_240911482/week6$ ./consumer_shm
Received word: hridini
4ITB2@debian:~/hridini_240911482/week6$
```

LAB NO: 7

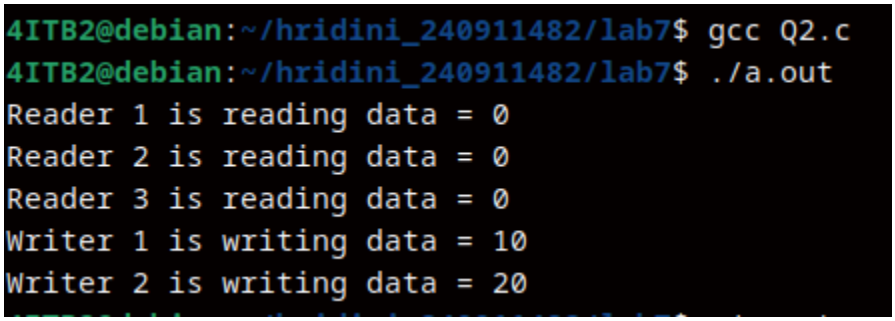
CLASSICAL PROBLEMS OF SYNCHRONIZATION

Lab Exercise

1. Modify the above Producer-Consumer program so that, a producer can produce at the most 10 items more than what the consumer has consumed.

```
4ITB2@debian:~/hridini_240911482/lab7$ gcc Q1.c
4ITB2@debian:~/hridini_240911482/lab7$ ./a.out
Produced: 0 | Total Produced: 1
Consumed: 0 | Total Consumed: 1
Produced: 1 | Total Produced: 2
Consumed: 1 | Total Consumed: 2
Produced: 2 | Total Produced: 3
Consumed: 2 | Total Consumed: 3
Produced: 3 | Total Produced: 4
Consumed: 3 | Total Consumed: 4
Produced: 4 | Total Produced: 5
Consumed: 4 | Total Consumed: 5
Produced: 5 | Total Produced: 6
Consumed: 5 | Total Consumed: 6
Produced: 6 | Total Produced: 7
Consumed: 6 | Total Consumed: 7
Produced: 7 | Total Produced: 8
Consumed: 7 | Total Consumed: 8
Produced: 8 | Total Produced: 9
Consumed: 8 | Total Consumed: 9
Produced: 9 | Total Produced: 10
Consumed: 9 | Total Consumed: 10
Produced: 10 | Total Produced: 11
Consumed: 10 | Total Consumed: 11
^C
```

2. Write a C/C++ program for first readers-writers problem using semaphores.



Q1.c Q2.c

```c
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

#define SIZE 5
#define MAX_ITEMS 20

int buf[SIZE];
int f = -1, r = -1;
int produced = 0;
int consumed = 0;
sem_t mutex, full, empty, limit;
void* produce(void* arg){
    for(int i = 0; i < MAX_ITEMS; i++) {
        sem_wait(&limit);     // ensure producer doesn't exceed 10 extra items
        sem_wait(&empty);
        sem_wait(&mutex);

        r = (r + 1) % SIZE;
        buf[r] = i;
        produced++;
        printf("Produced: %d | Total Produced: %d\n", i, produced);
        sem_post(&mutex);
        sem_post(&full);
        sleep(1);}
    return NULL;}

void* consume(void* arg){
    for(int i = 0; i < MAX_ITEMS; i++){
        sem_wait(&full);
        sem_wait(&mutex);

        f = (f + 1) % SIZE;
        int item = buf[f];
        consumed++;
        printf("Consumed: %d | Total Consumed: %d\n", item, consumed);
        sem_post(&mutex);
        sem_post(&empty);
        sem_post(&limit);   // allow producer to produce more
        sleep(1);}
    return NULL;}
```

```
44  int main(){
45      pthread_t t1, t2;
46      sem_init(&mutex, 0, 1);
47      sem_init(&full, 0, 0);
48      sem_init(&empty, 0, SIZE);
49      sem_init(&limit, 0, 10);   // producer can be ahead by at most 10
50      pthread_create(&t1, NULL, produce, NULL);
51      pthread_create(&t2, NULL, consume, NULL);
52      pthread_join(t1, NULL);
53      pthread_join(t2, NULL);
54      sem_destroy(&mutex);
55      sem_destroy(&full);
56      sem_destroy(&empty);
57      sem_destroy(&limit);
58      return 0;}
```

Q1.c Q2.c

```c
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>

sem_t mutex, wrt;
int readcount = 0;
int data = 0;

void* reader(void* arg){
    int id = *(int*)arg;
    sem_wait(&mutex);
    readcount++;
    if(readcount == 1)
        sem_wait(&wrt);   // first reader blocks writers
    sem_post(&mutex);
    printf("Reader %d is reading data = %d\n", id, data);
    sleep(1);
    sem_wait(&mutex);
    readcount--;
    if(readcount == 0)
        sem_post(&wrt);   // last reader allows writers
    sem_post(&mutex);
    return NULL;}

void* writer(void* arg){
    int id = *(int*)arg;
    sem_wait(&wrt);
    data += 10;
    printf("Writer %d is writing data = %d\n", id, data);
    sleep(1);
    sem_post(&wrt);
    return NULL;
}
```

```
36  int main(){
37      pthread_t r[3], w[2];
38      int r_id[3] = {1,2,3};
39      int w_id[2] = {1,2};
40
41      sem_init(&mutex, 0, 1);
42      sem_init(&wrt, 0, 1);
43
44      for(int i = 0; i < 3; i++)
45          pthread_create(&r[i], NULL, reader, &r_id[i]);
46
47      for(int i = 0; i < 2; i++)
48          pthread_create(&w[i], NULL, writer, &w_id[i]);
49
50      for(int i = 0; i < 3; i++)
51          pthread_join(r[i], NULL);
52
53      for(int i = 0; i < 2; i++)
54          pthread_join(w[i], NULL);
55
56      sem_destroy(&mutex);
57      sem_destroy(&wrt);
58      return 0;
59  }
```

LAB 8

DEADLOCK MANAGEMENT ALGORITHMS

Lab Exercises

1. Consider the following snapshot of the system. Write C/C++ program to implement Banker's
algorithm for deadlock avoidance. The program has to accept all inputs from the user. Assume
the total number of instances of A,B and C are 10,5 and 7 respectively.

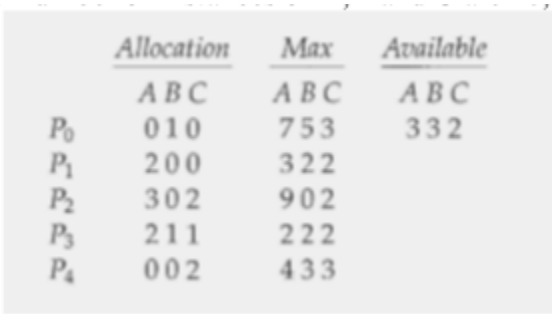
(a) What is the content of the matrix Need?

(b) Is the system in a safe state?

(c) If a request from process P1 arrives for (1, 0, 2), can the request be granted immediately?
Display the updated Allocation, Need and Available matrices.

(d) If a request from process P4 arrives for (3, 3, 0), can the request be granted immediately?

(e) If a request from process P0 arrives for (0, 2, 0), can the request be granted immediately?



	Allocation	*Max*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

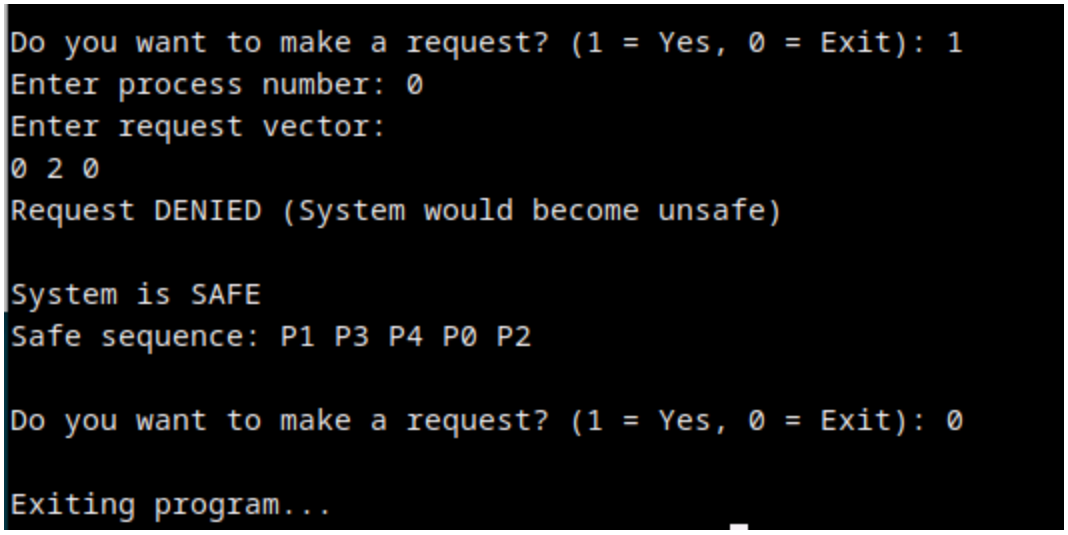
```c
#include <stdio.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[n][m], max[n][m], need[n][m], available[m];
    printf("\nEnter Allocation Matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("%d", &allocation[i][j]);
        } }

    printf("\nEnter Max Matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("%d", &max[i][j]);
        } }

    printf("\nEnter Available Resources:\n");
    for(int i=0;i<m;i++){
        scanf("%d", &available[i]);
    }
    // Calculate Need
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            need[i][j] = max[i][j] - allocation[i][j]; }  }
    while(1) {
        //  🔹 Safety Algorithm
        int finish[n], safeSeq[n], work[m];
        for(int i=0;i<n;i++) finish[i] = 0;
        for(int i=0;i<m;i++) work[i] = available[i];
        int count = 0;
```

```
        int count = 0;
        while(count < n){
            int found = 0;
            for(int i=0;i<n;i++){
                if(!finish[i]){
                    int j;
                    for(j=0;j<m;j++){
                        if(need[i][j] > work[j])
                            break; }
                    if(j == m){
                        for(int k=0;k<m;k++)
                            work[k] += allocation[i][k];

                        safeSeq[count++] = i;
                        finish[i] = 1;
                        found = 1;  }} }
            if(!found){
                printf("\nSystem is NOT in safe state\n");
                break;}}
        if(count == n){
            printf("\nSystem is SAFE\nSafe sequence: ");
            for(int i=0;i<n;i++)
                printf("P%d ", safeSeq[i]);
            printf("\n");   }
        //  • Ask for request
        int choice;
        printf("\nDo you want to make a request? (1 = Yes, 0 = Exit): ");
        scanf("%d", &choice);
        if(choice == 0)
            break;
        int p;
        printf("Enter process number: ");
        scanf("%d", &p);
        int request[m];
        printf("Enter request vector:\n");
        for(int i=0;i<m;i++){
            scanf("%d", &request[i]); }
        // Check Request <= Need
        int valid = 1;
        for(int i=0;i<m;i++){
            if(request[i] > need[p][i]){
                printf("Error: Exceeds maximum claim\n");
                valid = 0;
                break; }}
        if(!valid) continue;
```

```
while(1) {
        // Check Request <= Need
        int valid = 1;
        for(int i=0;i<m;i++){
            if(request[i] > need[p][i]){
                printf("Error: Exceeds maximum claim\n");
                valid = 0;
                break; }}
        if(!valid) continue;
        // Check Request <= Available
        for(int i=0;i<m;i++){
            if(request[i] > available[i]){
                printf("Resources not available, process must wait\n");
                valid = 0;
                break;
            }}
        if(!valid) continue;
        //  • Save old state (IMPORTANT)
        int temp_available[m], temp_alloc[n][m], temp_need[n][m];
        for(int i=0;i<m;i++)
            temp_available[i] = available[i];
        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                temp_alloc[i][j] = allocation[i][j];
                temp_need[i][j] = need[i][j]; }
        // Pretend allocation
        for(int i=0;i<m;i++){
            available[i] -= request[i];
            allocation[p][i] += request[i];
            need[p][i] -= request[i]; }
        //  • Check safety again
        for(int i=0;i<n;i++) finish[i] = 0;
        for(int i=0;i<m;i++) work[i] = available[i];
        count = 0;
        while(count < n){
            int found = 0;
            for(int i=0;i<n;i++){
                if(!finish[i]){
                    int j;
                    for(j=0;j<m;j++){
                        if(need[i][j] > work[j])
                            break; }
                    if(j == m){
                        for(int k=0;k<m;k++)
                            work[k] += allocation[i][k];
                        finish[i] = 1;
                        found = 1;
                        count++;}}  }
```

```
                        found = 1;
                        count++;}}  }
            if(!found) break;
        }
        if(count == n){
            printf("Request GRANTED (System is still safe)\n");
        } else {
            printf("Request DENIED (System would become unsafe)\n");
            // Restore old state
            for(int i=0;i<m;i++)
                available[i] = temp_available[i];
            for(int i=0;i<n;i++){
                for(int j=0;j<m;j++){
                    allocation[i][j] = temp_alloc[i][j];
                    need[i][j] = temp_need[i][j];  }  }  } }
    printf("\nExiting program...\n");
    return 0;
}
```

```
41TB2@debian:~/hridini_240911482/week8$ ./a.out
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

System is SAFE
Safe sequence: P1 P3 P4 P0 P2

Do you want to make a request? (1 = Yes, 0 = Exit): 1
Enter process number: 1
Enter request vector:
1 0 2
Request GRANTED (System is still safe)

System is SAFE
Safe sequence: P1 P3 P4 P0 P2

Do you want to make a request? (1 = Yes, 0 = Exit): 1
Enter process number: 4
Enter request vector:
3 3 0
Resources not available, process must wait

System is SAFE
Safe sequence: P1 P3 P4 P0 P2
```



2. Consider the following snapshot of the system. Write C/C++ program to implement deadlock
detection algorithm.
(a) Is the system in a safe state?
(b) Suppose that process P2 make one additional request for instance of type C, can the
system still be in a safe state?

```c
#include <stdio.h>

int main() {
    int n, m;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resource types: ");
    scanf("%d", &m);
    int allocation[n][m], request[n][m], available[m];
    printf("\nEnter Allocation Matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("%d", &allocation[i][j]); }  }

    printf("\nEnter Request Matrix:\n");
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++){
            scanf("%d", &request[i][j]);} }

    printf("\nEnter Available Resources:\n");
    for(int i=0;i<m;i++){
        scanf("%d", &available[i]); }

    int finish[n], work[m];

    // Initialize finish
    for(int i=0;i<n;i++){
        int zero = 1;
        for(int j=0;j<m;j++){
            if(allocation[i][j] != 0){
                zero = 0;
                break;
            }}
        finish[i] = zero; }

    for(int i=0;i<m;i++)
        work[i] = available[i];

    int changed = 1;
```

```
    int changed = 1;

    while(changed){
        changed = 0;

        for(int i=0;i<n;i++){
            if(!finish[i]){
                int j;
                for(j=0;j<m;j++){
                    if(request[i][j] > work[j])
                        break;}
                if(j == m){
                    for(int k=0;k<m;k++)
                        work[k] += allocation[i][k];

                    finish[i] = 1;
                    changed = 1; }}}}

    int deadlock = 0;
    printf("\n");
    for(int i=0;i<n;i++){
        if(!finish[i]){
            printf("P%d is deadlocked\n", i);
            deadlock = 1; }}
    if(!deadlock)
        printf("No deadlock detected\n");
    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week8$ vi q2.c
4ITB2@debian:~/hridini_240911482/week8$ gcc q2.c
4ITB2@debian:~/hridini_240911482/week8$ ./a.out
Enter number of processes: 5
Enter number of resource types: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
0 0 0

No deadlock detected
```

	Allocation	*Request*	*Available*
	A B C	*A B C*	*A B C*
P_0	0 1 0	0 0 0	0 0 0
P_1	2 0 0	2 0 2	
P_2	3 0 3	0 0 0	
P_3	2 1 1	1 0 0	
P_4	0 0 2	0 0 2	

LAB 9

MEMORY MANAGEMENT I

Lab Exercises:
1. Write a C/C++ program to simulate First-fit, Best-fit and Worst-fit strategies. Given memory
partitions of 100K, 500K, 200K, 300K, and 600K(in order), how would each of the First-fit,
Best-fit, and Worst-fit algorithms place processes of 212K, 417K, 112K, and 426K (in
order)? Which algorithm makes efficient use of memory?

Q1.c

```c
#include <stdio.h>
#include <string.h>

#define MAX 10
void firstFit(int blocks[], int nb, int procs[], int np) {
    int alloc[MAX];
    int temp[MAX];
    memset(alloc, -1, sizeof(alloc));
    for (int i = 0; i < nb; i++) temp[i] = blocks[i];

    printf("\n--- First Fit ---\n");
    for (int i = 0; i < np; i++) {
        for (int j = 0; j < nb; j++) {
            if (temp[j] >= procs[i]) {
                alloc[i] = j;
                temp[j] -= procs[i];
                break;
            }
        }
        if (alloc[i] != -1)
            printf("Process %d (%dK) -> Block %d (%dK)\n", i+1, procs[i], alloc[i]+1, blocks[alloc[i]]);
        else
            printf("Process %d (%dK) -> Not Allocated\n", i+1, procs[i]); }}

void bestFit(int blocks[], int nb, int procs[], int np) {
    int alloc[MAX];
    int temp[MAX];
    memset(alloc, -1, sizeof(alloc));
    for (int i = 0; i < nb; i++) temp[i] = blocks[i];
    printf("\n--- Best Fit ---\n");
    for (int i = 0; i < np; i++) {
        int best = -1;
        for (int j = 0; j < nb; j++) {
            if (temp[j] >= procs[i]) {
                if (best == -1 || temp[j] < temp[best])
                    best = j;
            }}
        alloc[i] = best;
        if (best != -1) {
            temp[best] -= procs[i];
            printf("Process %d (%dK) -> Block %d (%dK)\n", i+1, procs[i], best+1, blocks[best]);
        } else {
            printf("Process %d (%dK) -> Not Allocated\n", i+1, procs[i]); }}}
```

```
        } else {
            printf("Process %d (%dK) -> Not Allocated\n", i+1, procs[i]); }}}

void worstFit(int blocks[], int nb, int procs[], int np) {
    int alloc[MAX];
    int temp[MAX];
    memset(alloc, -1, sizeof(alloc));
    for (int i = 0; i < nb; i++) temp[i] = blocks[i];

    printf("\n--- Worst Fit ---\n");
    for (int i = 0; i < np; i++) {
        int worst = -1;
        for (int j = 0; j < nb; j++) {
            if (temp[j] >= procs[i]) {
                if (worst == -1 || temp[j] > temp[worst])
                    worst = j;
            }}
        alloc[i] = worst;
        if (worst != -1) {
            temp[worst] -= procs[i];
            printf("Process %d (%dK) -> Block %d (%dK)\n", i+1, procs[i], worst+1, blocks[worst]);}
        else {
            printf("Process %d (%dK) -> Not Allocated\n", i+1, procs[i]);
        }}}

int main() {
    int blocks[] = {100, 500, 200, 300, 600};
    int procs[]  = {212, 417, 112, 426};
    int nb = 5, np = 4;

    printf("Memory Blocks (K): ");
    for (int i = 0; i < nb; i++) printf("%d ", blocks[i]);
    printf("\nProcesses (K):     ");
    for (int i = 0; i < np; i++) printf("%d ", procs[i]);

    firstFit(blocks, nb, procs, np);
    bestFit(blocks, nb, procs, np);
    worstFit(blocks, nb, procs, np);

    printf("\nConclusion: Best-fit makes most efficient use of memory by minimizing leftover holes.\n");
    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week9$ gcc Q1.c
4ITB2@debian:~/hridini_240911482/week9$ ./a.out
Memory Blocks (K): 100 500 200 300 600
Processes (K):     212 417 112 426
--- First Fit ---
Process 1 (212K) -> Block 2 (500K)
Process 2 (417K) -> Block 5 (600K)
Process 3 (112K) -> Block 2 (500K)
Process 4 (426K) -> Not Allocated

--- Best Fit ---
Process 1 (212K) -> Block 4 (300K)
Process 2 (417K) -> Block 2 (500K)
Process 3 (112K) -> Block 3 (200K)
Process 4 (426K) -> Block 5 (600K)

--- Worst Fit ---
Process 1 (212K) -> Block 5 (600K)
Process 2 (417K) -> Block 2 (500K)
Process 3 (112K) -> Block 5 (600K)
Process 4 (426K) -> Not Allocated

Conclusion: Best-fit makes most efficient use of memory by minimizing leftover holes.
```

2. Assuming a page size of 32 bytes and there are total of 8 such pages totaling 256 bytes. Write
a C program to simulate this memory mapping. The program should read the logical memory
address and display the page number and page offset in decimal. How many bytes do you
need to represent the address in this scenario? Display the page number and offset to
reference the following logical addresses.
(i) 204 byte
(ii) 56 byte

C Q2.c

```c
#include <stdio.h>
#include <math.h>

int main() {
    int page_size = 32;   // bytes
    int num_pages = 8;
    int total_mem = page_size * num_pages;  // 256 bytes

    // Number of bits needed: log2(256) = 8 bits
    int total_bits  = (int)(log2(total_mem));   // 8 bits
    int offset_bits = (int)(log2(page_size));   // 5 bits (2^5 = 32)
    int page_bits   = total_bits - offset_bits; // 3 bits

    printf("Page Size    : %d bytes\n", page_size);
    printf("Total Pages  : %d\n", num_pages);
    printf("Total Memory : %d bytes\n", total_mem);
    printf("Address bits : %d bits total (%d page bits + %d offset bits)\n\n",
           total_bits, page_bits, offset_bits);

    int addresses[] = {204, 56};
    int n = 2;

    printf("%-20s %-15s %-15s\n", "Logical Address", "Page Number", "Page Offset");
    printf("-------------------------------------------------------\n");

    for (int i = 0; i < n; i++) {
        int addr   = addresses[i];
        int page   = addr / page_size;
        int offset = addr % page_size;
        printf("%-20d %-15d %-15d\n", addr, page, offset);
    }

    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week9$ gcc Q2.c -lm
4ITB2@debian:~/hridini_240911482/week9$ ./a.out
Page Size    : 32 bytes
Total Pages  : 8
Total Memory : 256 bytes
Address bits : 8 bits total (3 page bits + 5 offset bits)

Logical Address      Page Number     Page Offset
-------------------------------------------------------
204                  6               12
56                   1               24
```

LAB 10

MEMORY MANAGEMENT II

Lab exercises :

1. Write a C/C++ program to simulate page replacement algorithms: FIFO and optimal. Frame allocation has to be done as per user input and use dynamic allocation for all data structures.

Q1.c

```c
#include <stdio.h>
#include <stdlib.h>

// Check if page is already in frames
int isPresent(int *frames, int nf, int page) {
    for (int i = 0; i < nf; i++)
        if (frames[i] == page) return 1;
    return 0;}

// FIFO Page Replacement
void fifo(int *pages, int np, int nf) {
    int *frames = (int *)malloc(nf * sizeof(int));
    int faults = 0, pointer = 0;

    for (int i = 0; i < nf; i++) frames[i] = -1;

    printf("\n--- FIFO Page Replacement ---\n");
    printf("%-10s %-20s %-10s\n", "Page", "Frames", "Fault");
    for (int i = 0; i < np; i++) {
        printf("%-10d ", pages[i]);

        if (!isPresent(frames, nf, pages[i])) {
            frames[pointer] = pages[i];
            pointer = (pointer + 1) % nf;
            faults++;

            for (int j = 0; j < nf; j++)
                if (frames[j] != -1) printf("%d ", frames[j]);
            printf("\t\tFault\n");}
        else {
            for (int j = 0; j < nf; j++)
                if (frames[j] != -1) printf("%d ", frames[j]);
            printf("\t\tNo Fault\n");}}

    printf("\nTotal Page Faults (FIFO) = %d\n", faults);
    printf("Hit Ratio = %.2f%%\n", ((float)(np - faults) / np) * 100);
    free(frames);}
```

```
    free(frames);}

// Find optimal victim: page used farthest in future
int findOptimal(int *frames, int nf, int *pages, int np, int current) {
    int farthest = -1, victim = 0;

    for (int i = 0; i < nf; i++) {
        int j;
        for (j = current + 1; j < np; j++) {
            if (frames[i] == pages[j]) {
                if (j > farthest) {
                    farthest = j;
                    victim = i; }
                break;}}
        // Page not used in future at all – best to replace
        if (j == np) return i; }
    return victim;}

// Optimal Page Replacement
```

```
// Optimal Page Replacement
void optimal(int *pages, int np, int nf) {
    int *frames = (int *)malloc(nf * sizeof(int));
    int faults = 0, filled = 0;

    for (int i = 0; i < nf; i++) frames[i] = -1;

    printf("\n--- Optimal Page Replacement ---\n");
    printf("%-10s %-20s %-10s\n", "Page", "Frames", "Fault");
    for (int i = 0; i < np; i++) {
        printf("%-10d ", pages[i]);

        if (!isPresent(frames, nf, pages[i])) {
            if (filled < nf) {
                frames[filled++] = pages[i];
            } else {
                int victim = findOptimal(frames, nf, pages, np, i);
                frames[victim] = pages[i];
            }
            faults++;

            for (int j = 0; j < nf; j++)
                if (frames[j] != -1) printf("%d ", frames[j]);
            printf("\t\tFault\n");}
        else {
            for (int j = 0; j < nf; j++)
                if (frames[j] != -1) printf("%d ", frames[j]);
            printf("\t\tNo Fault\n"); }}

    printf("\nTotal Page Faults (Optimal) = %d\n", faults);
    printf("Hit Ratio = %.2f%%\n", ((float)(np - faults) / np) * 100);
    free(frames);}
```

```c
int main() {
    int np, nf;

    printf("Enter number of frames: ");
    scanf("%d", &nf);

    printf("Enter number of pages: ");
    scanf("%d", &np);

    int *pages = (int *)malloc(np * sizeof(int));

    printf("Enter page reference string:\n");
    for (int i = 0; i < np; i++)
        scanf("%d", &pages[i]);

    fifo(pages, np, nf);
    optimal(pages, np, nf);

    free(pages);
    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week10$ gcc Q1.c
4ITB2@debian:~/hridini_240911482/week10$ ./a.out
Enter number of frames: 3
Enter number of pages: 20
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

--- FIFO Page Replacement ---
Page       Frames               Fault
7          7            Fault
0          7 0          Fault
1          7 0 1                Fault
2          2 0 1                Fault
0          2 0 1                No Fault
3          2 3 1                Fault
0          2 3 0                Fault
4          4 3 0                Fault
2          4 2 0                Fault
3          4 2 3                Fault
0          0 2 3                Fault
3          0 2 3                No Fault
2          0 2 3                No Fault
1          0 1 3                Fault
2          0 1 2                Fault
0          0 1 2                No Fault
1          0 1 2                No Fault
7          7 1 2                Fault
0          7 0 2                Fault
1          7 0 1                Fault

Total Page Faults (FIFO) = 15
Hit Ratio = 25.00%
```

```
Hit Ratio = 25.00%

--- Optimal Page Replacement ---
Page       Frames                Fault
7          7             Fault
0          7 0           Fault
1          7 0 1                 Fault
2          2 0 1                 Fault
0          2 0 1                 No Fault
3          2 0 3                 Fault
0          2 0 3                 No Fault
4          2 4 3                 Fault
2          2 4 3                 No Fault
3          2 4 3                 No Fault
0          2 0 3                 Fault
3          2 0 3                 No Fault
2          2 0 3                 No Fault
1          2 0 1                 Fault
2          2 0 1                 No Fault
0          2 0 1                 No Fault
1          2 0 1                 No Fault
7          7 0 1                 Fault
0          7 0 1                 No Fault
1          7 0 1                 No Fault

Total Page Faults (Optimal) = 9
Hit Ratio = 55.00%
4IIB2@debian:~/bridipi_240911482/week10$ gcc
```

2. Write a C/C++ program to simulate LRU Page Replacement. Frame allocation must be done
as per user input and dynamic allocation for all data structures. Find the total number of page
faults and hit ratio for the algorithm.

```c
#include <stdio.h>
#include <stdlib.h>

int isPresent(int *frames, int nf, int page) {
    for (int i = 0; i < nf; i++)
        if (frames[i] == page) return 1;
    return 0;
}

// Find LRU victim: page used least recently
int findLRU(int *time, int nf) {
    int min = time[0], pos = 0;
    for (int i = 1; i < nf; i++) {
        if (time[i] < min) {
            min = time[i];
            pos = i;}}
    return pos;}

void lru(int *pages, int np, int nf) {
    int *frames = (int *)malloc(nf * sizeof(int));
    int *time   = (int *)malloc(nf * sizeof(int));
    int faults  = 0, filled = 0;

    for (int i = 0; i < nf; i++) {
        frames[i] = -1;
        time[i]   = 0;}

    printf("\n--- LRU Page Replacement ---\n");
    printf("%-10s %-25s %-10s\n", "Page", "Frames", "Fault");
    for (int i = 0; i < np; i++) {
        printf("%-10d ", pages[i]);

        if (!isPresent(frames, nf, pages[i])) {
            int pos;
            if (filled < nf) {
                pos = filled++;
            } else {
                pos = findLRU(time, nf);}
            frames[pos] = pages[i];
            time[pos]   = i + 1;
            faults++;
```

```
            time[pos]   = i + 1;
            faults++;

            for (int j = 0; j < nf; j++)
                if (frames[j] != -1) printf("%d ", frames[j]);}
        else {
            // Update time for the referenced page
            for (int j = 0; j < nf; j++)
                if (frames[j] == pages[i])
                    time[j] = i + 1;

            for (int j = 0; j < nf; j++)
                if (frames[j] != -1) printf("%d ", frames[j]);
            printf("\t\tNo Fault\n");
        }}

    int hits = np - faults;
    printf("\nTotal Page Faults = %d\n", faults);
    printf("Total Page Hits   = %d\n", hits);
    printf("Hit Ratio         = %.2f%%\n", ((float)hits / np) * 100);
    free(frames);
    free(time);}

int main() {
    int np, nf;

    printf("Enter number of frames: ");
    scanf("%d", &nf);

    printf("Enter number of pages: ");
    scanf("%d", &np);

    int *pages = (int *)malloc(np * sizeof(int));

    printf("Enter page reference string:\n");
    for (int i = 0; i < np; i++)
        scanf("%d", &pages[i]);

    lru(pages, np, nf);
    free(pages);
    return 0;
}
```

```
Hit Ratio = 55.00%
4ITB2@debian:~/hridini_240911482/week10$ gcc Q2.c
4ITB2@debian:~/hridini_240911482/week10$ ./a.out
Enter number of frames: 3
Enter number of pages: 20
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1

--- LRU Page Replacement ---
Page       Frames                    Fault
7          7           Fault
0          7 0         Fault
1          7 0 1                  Fault
2          2 0 1                  Fault
0          2 0 1                  No Fault
3          2 0 3                  Fault
0          2 0 3                  No Fault
4          4 0 3                  Fault
2          4 0 2                  Fault
3          4 3 2                  Fault
0          0 3 2                  Fault
3          0 3 2                  No Fault
2          0 3 2                  No Fault
1          1 3 2                  Fault
2          1 3 2                  No Fault
0          1 0 2                  Fault
1          1 0 2                  No Fault
7          1 0 7                  Fault
0          1 0 7                  No Fault
1          1 0 7                  No Fault

Total Page Faults = 12
Total Page Hits   = 8
Hit Ratio         = 40.00%
4ITB2@debian:~/hridini_240911482/week10$
```

LAB 11

DISK SCHEDULING ALGORITHM

Lab Exercises:
1. Write a C/C++ Program to simulate the following algorithms find the total no. of cylinder
movements for various input requests
(i) FCFS (ii) SSTF (iii) SCAN
(iv) C-SCAN

Additional Exercises:
1. Write a C/C++ Program to simulate the following algorithms find the total no. of cylinder
movements for various input requests
(i) LOOK (ii) C-LOOK

C full.c

```c

#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX 100
#define DISK_SIZE 200   /* cylinders 0 to 199 */

/* ── utility ── */
int cmp(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int absVal(int x) { return x < 0 ? -x : x; }

/* ═══════════════════════════════════════════════════
 * Q1 - Algorithm 1: FCFS
 * ═══════════════════════════════════════════════════ */
void fcfs(int queue[], int n, int head) {
    int thm = 0, cur = head;
    printf("\n--- FCFS ---\n");
    printf("Sequence: %d", head);
    for (int i = 0; i < n; i++) {
        thm += absVal(queue[i] - cur);
        cur = queue[i];
        printf(" -> %d", cur);
    }
    printf("\nTotal Head Movement: %d tracks\n", thm);
}
```

```c
/* ═══════════════════════════════════════════════════
 * Q1 - Algorithm 2: SSTF
 * ═══════════════════════════════════════════════════ */
void sstf(int queue[], int n, int head) {
    int visited[MAX] = {0};
    int thm = 0, cur = head;
    printf("\n--- SSTF ---\n");
    printf("Sequence: %d", head);
    for (int count = 0; count < n; count++) {
        int minDist = 99999, idx = -1;
        for (int i = 0; i < n; i++) {
            if (!visited[i] && absVal(queue[i] - cur) < minDist) {
                minDist = absVal(queue[i] - cur);
                idx = i;
            }
        }
        visited[idx] = 1;
        thm += minDist;
        cur = queue[idx];
        printf(" -> %d", cur);
    }
    printf("\nTotal Head Movement: %d tracks\n", thm);
}

/* ═══════════════════════════════════════════════════
 * Q1 - Algorithm 3: SCAN (Elevator)
 * direction: 0 = toward 0, 1 = toward max
 * ═══════════════════════════════════════════════════ */
void scan(int queue[], int n, int head, int direction) {
    int sorted[MAX], thm = 0, cur = head;
    for (int i = 0; i < n; i++) sorted[i] = queue[i];
    qsort(sorted, n, sizeof(int), cmp);

    printf("\n--- SCAN (direction: %s) ---\n", direction ? "toward max" : "toward 0");
    printf("Sequence: %d", head);

    if (direction == 0) {
        /* go left first, hit 0, then go right */
        for (int i = n - 1; i >= 0; i--) {
            if (sorted[i] <= head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
```

```
        }
        /* go to 0 */
        thm += cur;
        cur = 0;
        printf(" -> 0");
        for (int i = 0; i < n; i++) {
            if (sorted[i] > head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
    } else {
        /* go right first, hit max, then go left */
        for (int i = 0; i < n; i++) {
            if (sorted[i] >= head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
        /* go to max (DISK_SIZE - 1) */
        thm += absVal((DISK_SIZE - 1) - cur);
        cur = DISK_SIZE - 1;
        printf(" -> %d", cur);
        for (int i = n - 1; i >= 0; i--) {
            if (sorted[i] < head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
    }
    printf("\nTotal Head Movement: %d tracks\n", thm);
}
```

full.c

```c
/* ═══════════════════════════════════════════════════
 * Q1 - Algorithm 4: C-SCAN
 * ═══════════════════════════════════════════════════ */
void cscan(int queue[], int n, int head) {
    int sorted[MAX], thm = 0, cur = head;
    for (int i = 0; i < n; i++) sorted[i] = queue[i];
    qsort(sorted, n, sizeof(int), cmp);
    printf("\n--- C-SCAN ---\n");
    printf("Sequence: %d", head);
    /* serve requests >= head going right */
    for (int i = 0; i < n; i++) {
        if (sorted[i] >= head) {
            thm += absVal(sorted[i] - cur);
            cur = sorted[i];
            printf(" -> %d", cur);}}
    /* jump to max end (not counted in THM) */
    printf(" -> %d(end)", DISK_SIZE - 1);
    cur = DISK_SIZE - 1;
    /* jump to 0 (not counted in THM) */
    printf(" -> 0(start)");
    cur = 0;
    /* serve remaining requests from beginning */
    for (int i = 0; i < n; i++) {
        if (sorted[i] < head) {
            thm += absVal(sorted[i] - cur);
            cur = sorted[i];
            printf(" -> %d", cur);}}
    printf("\nTotal Head Movement: %d tracks\n", thm);
    printf("(Note: jumps from end to start are not counted in THM)\n");
}
/* ═══════════════════════════════════════════════════
 * Q2 - Algorithm 5: LOOK
 * ═══════════════════════════════════════════════════ */
void look(int queue[], int n, int head, int direction) {
    int sorted[MAX], thm = 0, cur = head;
    for (int i = 0; i < n; i++) sorted[i] = queue[i];
    qsort(sorted, n, sizeof(int), cmp);

    printf("\n--- LOOK (direction: %s) ---\n", direction ? "toward max" : "toward 0");
    printf("Sequence: %d", head);

    if (direction == 0) {
        /* go left to farthest request, then right */
        for (int i = n - 1; i >= 0; i--) {
            if (sorted[i] <= head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
```

```c
    } else {
        /* go right to farthest request, then left */
        for (int i = 0; i < n; i++) {
            if (sorted[i] >= head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
        for (int i = n - 1; i >= 0; i--) {
            if (sorted[i] < head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
    }
    printf("\nTotal Head Movement: %d tracks\n", thm);
}

/* ═══════════════════════════════════════════════
 * Q2 - Algorithm 6: C-LOOK
 * ═══════════════════════════════════════════════ */
void clook(int queue[], int n, int head) {
    int sorted[MAX], thm = 0, cur = head;
    for (int i = 0; i < n; i++) sorted[i] = queue[i];
    qsort(sorted, n, sizeof(int), cmp);

    printf("\n--- C-LOOK ---\n");
    printf("Sequence: %d", head);

    /* serve requests >= head going right */
    for (int i = 0; i < n; i++) {
        if (sorted[i] >= head) {
            thm += absVal(sorted[i] - cur);
            cur = sorted[i];
            printf(" -> %d", cur);
        }
    }
```

```c
    /* jump back to smallest request (not counted) */
    if (sorted[0] < head) {
        printf(" -> %d(jump)", sorted[0]);
        cur = sorted[0];
        /* serve remaining from beginning */
        for (int i = 1; i < n; i++) {
            if (sorted[i] < head) {
                thm += absVal(sorted[i] - cur);
                cur = sorted[i];
                printf(" -> %d", cur);
            }
        }
    }
    printf("\nTotal Head Movement: %d tracks\n", thm);
    printf("(Note: circular jump is not counted in THM)\n");
}

/* ═══════════════════════════════════════════════
 * MAIN
 * ═══════════════════════════════════════════════ */
int main() {
    int queue[MAX], n, head, choice, dir;

    printf("=== Disk Scheduling Simulator ===\n");
    printf("Disk cylinders: 0 to %d\n\n", DISK_SIZE - 1);

    printf("Enter number of requests: ");
    scanf("%d", &n);

    printf("Enter the request queue:\n");
    for (int i = 0; i < n; i++) {
        printf("  Request %d: ", i + 1);
        scanf("%d", &queue[i]);
    }

    printf("Enter current head position: ");
    scanf("%d", &head);

    printf("\nSelect Algorithm:\n");
    printf("  --- Q1 ---\n");
    printf("  1. FCFS\n");
    printf("  2. SSTF\n");
    printf("  3. SCAN\n");
    printf("  4. C-SCAN\n");
    printf("  --- Q2 ---\n");
    printf("  5. LOOK\n");
    printf("  6. C-LOOK\n");
    printf("  7. Run ALL\n");
```

```c
    printf("  7. Run ALL\n");
    printf("Choice: ");
    scanf("%d", &choice);

    if (choice == 3 || choice == 5) {
        printf("Initial direction (0 = toward 0, 1 = toward max): ");
        scanf("%d", &dir);
    }

    switch (choice) {
        case 1: fcfs(queue, n, head); break;
        case 2: sstf(queue, n, head); break;
        case 3: scan(queue, n, head, dir); break;
        case 4: cscan(queue, n, head); break;
        case 5: look(queue, n, head, dir); break;
        case 6: clook(queue, n, head); break;
        case 7:
            fcfs(queue, n, head);
            sstf(queue, n, head);
            scan(queue, n, head, 0);
            cscan(queue, n, head);
            look(queue, n, head, 0);
            clook(queue, n, head);
            break;
        default:
            printf("Invalid choice.\n");
    }

    return 0;
}
```

```
4ITB2@debian:~/hridini_240911482/week11$ gcc full.c
4ITB2@debian:~/hridini_240911482/week11$ ./a.out
=== Disk Scheduling Simulator ===
Disk cylinders: 0 to 199

Enter number of requests: 8
Enter the request queue:
  Request 1: 98
  Request 2: 183
  Request 3: 37
  Request 4: 122
  Request 5: 14
  Request 6: 124
  Request 7: 65
  Request 8: 67
Enter current head position: 53

Select Algorithm:
  --- Q1 ---
  1. FCFS
  2. SSTF
  3. SCAN
  4. C-SCAN
  --- Q2 ---
  5. LOOK
  6. C-LOOK
  7. Run ALL
Choice: 7

--- FCFS ---
Sequence: 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67
Total Head Movement: 640 tracks

--- SSTF ---
Sequence: 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183
Total Head Movement: 236 tracks

--- SCAN (direction: toward 0) ---
Sequence: 53 -> 37 -> 14 -> 0 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183
Total Head Movement: 236 tracks

--- C-SCAN ---
Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199(end) -> 0(start) -> 14 -> 37
Total Head Movement: 167 tracks
(Note: jumps from end to start are not counted in THM)
```

```
--- C-SCAN ---
Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 199(end) -> 0(start) -> 14 -> 37
Total Head Movement: 167 tracks
(Note: jumps from end to start are not counted in THM)

--- LOOK (direction: toward 0) ---
Sequence: 53 -> 37 -> 14 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183
Total Head Movement: 208 tracks

--- C-LOOK ---
Sequence: 53 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183 -> 14(jump) -> 37
Total Head Movement: 153 tracks
(Note: circular jump is not counted in THM)
```